

DESIGN AND PERFORMANCE EVALUATION OF A ZEPHYR RTOS-BASED BLE
TEMPERATURE MONITORING NODE ON THE NRF52840 DEVELOPMENT KIT

By

Oluwaferanmi Arowoshola

A Research Project Submitted to the

ECET Department at Minnesota State University, Mankato

In Partial Fulfillment of the Master of Science Degree Requirements

In

Electrical and Computer Engineering

Under the Supervision of Dr. He Nannan

Mankato, Minnesota

May 2026

May 5, 2026

Design and Performance Evaluation of a Zephyr RTOS-Based BLE Temperature Monitoring
Node on the nRF52840 Development Kit

Oluwaferanmi Arowoshola

This research project is being presented to

Dr. Nannan He
Supervisor

ABSTRACT

This research project presents the development and performance evaluation of a Zephyr RTOS-based embedded wireless sensing node implemented on the Nordic nRF52840 Development Kit. The project began with establishment of a complete Zephyr development environment on a Windows host system, followed by verification of successful build, flash, and runtime execution on the target hardware. After board validation, the work progressed into practical exploration of Zephyr kernel behavior through multithreading, synchronization, priority scheduling, starvation, and fairness experiments executed directly on the nRF52840 DK.

Following the RTOS study, Bluetooth Low Energy (BLE) functionality was confirmed using a Zephyr peripheral sample and a mobile client application. A custom BLE sensing node was then developed, incorporating a simulated temperature source, a dedicated sensor thread, a custom GATT service, and a readable BLE characteristic. The final prototype advertised successfully as a BLE peripheral, connected to nRF Connect on a mobile phone, and returned changing characteristic values that reflected live application-side data updates.

Experimental evaluation focused on timing performance. Application-level latency was measured as the time between the most recent sensor update and a BLE characteristic read by the client. Scheduler timing was also measured under both baseline and interference conditions. Under unloaded conditions, the 2-second sensor thread remained highly stable, with an observed period of approximately 2007–2008 ms. Under interference from a higher-priority busy thread, the same periodic task showed a large but repeatable timing shift, with the average period increasing to approximately 2380 ms. These results show that the Zephyr scheduler maintained stable execution behavior while still exhibiting clear sensitivity to sustained higher-priority CPU contention.

A power-consumption experiment using the Nordic Power Profiler Kit II was also tried. Although measurement activity and traces were seen, the resulting current values were not considered trustworthy enough for formal quantitative reporting, and that part of the study was documented as incomplete. Despite that limitation, the project successfully shows the design, implementation, and timing characterization of a Zephyr RTOS-based BLE sensing node on embedded hardware. The work sets up a practical foundation for future extensions involving refined BLE data handling, physical sensor integration, reliable power profiling, and cloud-connected IoT gateway functionality.

TABLE OF CONTENTS

Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope of Work	3
1.5 Contributions	4
1.6 Organization of the Report	5
Chapter 2: Literature Review and Zephyr RTOS Overview	6
2.1 Introduction	6
2.2 Embedded Real-Time Operating Systems	6
2.3 Zephyr RTOS	7
2.4 Bluetooth Low Energy in Embedded IoT Systems	8
2.5 Embedded Performance Evaluation Metrics	9
2.6 Summary	10
Chapter 3: System Design	11
3.1 System Overview	11
3.2 Development Platform and Hardware	12
3.3 Software Architecture	14
3.4 Threading and Scheduling Design	16
3.5 BLE Communication Design	18
3.6 Sensor Data Design	20
3.7 Experimental Design Strategy	21
3.8 Summary	23
Chapter 4: Implementation	24
4.1 Introduction	24
4.2 Development Environment Setup	24
4.2.1 Host environment and workspace	24
4.2.2 Python compatibility resolution	25
4.2.3 Board target correction	25

4.2.4 SDK and toolchain integration.....	25
4.2.5 Build and flash pipeline verification.....	26
4.3 Board Validation and Initial Zephyr Application Work	26
4.4 Zephyr Threading and Synchronization Implementation	27
4.4.1 Multithreaded execution	27
4.4.2 Shared UART output and mutex protection.....	27
4.4.3 Priority experiments and starvation	28
4.4.4 Equal-priority case	28
4.5 BLE Validation Implementation	30
4.5.1 Stock BLE sample build path	30
4.5.2 BLE runtime failure and overlay fix.....	30
4.5.3 Mobile validation	30
4.6 Custom BLE Temperature Node Implementation	33
4.6.1 Project files	33
4.6.2 Initial custom application structure.....	33
4.6.3 Compile-time corrections.....	33
4.6.4 Mobile-side verification of custom node	34
4.7 Timing Instrumentation Implementation	37
4.7.1 Latency instrumentation.....	37
4.7.2 Baseline scheduler timing instrumentation.....	37
4.7.3 Interference implementation	37
4.8 Data Logging and Graph Preparation	37
4.9 Attempted Power Measurement Implementation.....	38
4.10 Design Decisions and Trade-Offs	38
4.11 Summary	39
Chapter 5: Experimental Results and Discussion	40
5.1 Experimental Setup.....	40
5.2 Thread Scheduling Experiments from Phase 2	41
5.2.1 Basic periodic thread execution	41
5.2.2 Shared UART output and synchronization	41
5.2.3 Starvation under a non-blocking high-priority thread	42

5.2.4 Yielding versus blocking.....	42
5.2.5 Equal-priority fairness	42
5.3 Task Latency Evaluation.....	45
5.3.1 Measurement method.....	45
5.3.2 Recorded latency data	45
5.3.3 Statistical summary	46
5.3.4 Interpretation.....	47
5.4 Scheduler Timing Under Baseline Conditions.....	48
5.4.1 Measurement method.....	48
5.4.2 Recorded baseline behavior	48
5.4.3 Baseline summary	49
5.4.4 Interpretation.....	49
5.5 Scheduler Timing Under Interference.....	50
5.5.1 Measurement method.....	50
5.5.2 Recorded interference behavior	50
5.5.3 Interference summary	50
5.5.4 Interpretation.....	51
5.6 Baseline vs Interference Comparison	51
5.6.1 Comparison table	51
5.6.2 Interpretation.....	51
5.7 Attempted Power Measurement.....	53
5.7.1 Attempted measurement setup	53
5.7.2 Why the data was rejected	53
5.7.3 Outcome of the Power Measurement Attempt.....	54
5.8 Discussion of Results.....	54
5.8.1 The custom node is functionally valid and experimentally measurable	54
5.8.2 Application-level latency is strongly shaped by periodic update timing	55
5.8.3 Zephyr periodic scheduling was highly stable under baseline conditions	55
5.8.4 CPU contention produced delay more than disorder	55
5.8.5 The project is strong in timing results, weaker in power results	56
5.9 Limitations	56

5.9.1 No validated power-consumption dataset	56
5.9.2 Simulated temperature source rather than physical sensor	56
5.9.3 Read-based BLE evaluation rather than streaming/notify behavior	56
5.9.4 Limited sample counts	57
5.9.5 Single hardware platform.....	57
5.10 Summary	57
Chapter 6: Conclusion, Limitations, and Future Work	58
6.1 Conclusion	58
6.2 Limitations	59
6.2.1 Incomplete power-consumption evaluation	59
6.2.2 Simulated sensing rather than physical sensor integration	60
6.2.3 BLE data model limited to read-based interaction	60
6.2.4 Limited scope of timing experiments	60
6.2.5 No completed smartphone-to-cloud gateway implementation	60
6.2.6 Platform-specific conclusions.....	60
6.3 Future Work	61
6.3.1 Reliable power-consumption measurement	61
6.3.2 Physical sensor integration	61
6.3.3 BLE notifications or indications	61
6.3.4 Gateway and cloud integration	61
6.3.5 Broader scheduler experimentation	62
6.3.6 Cross-platform comparison.....	62
6.4 Final Remarks	62
References	64
Appendices.....	66
Appendix A: Phase 2 Scheduler Experiment Code.....	66
Appendix B: Custom BLE Temperature Node Source Code.....	69
Appendix C: Raw Timing Logs	73
Appendix C.1:	73
Appendix C.2:	77
Appendix C.3:	83

Appendix D: Additional Screenshots.....	94
Appendix D.1:.....	94
Appendix D.2:.....	95
Appendix D.3:.....	96
Appendix D.4:.....	97
Appendix D.5:.....	98
Appendix D.6:.....	99
Appendix D.7:.....	100

Chapter 1: Introduction

1.1 Background

Real-time operating systems play a significant role in modern embedded systems because they provide structured task scheduling, deterministic or near-deterministic execution behavior, and efficient management of limited hardware resources. In wireless sensing and Internet of Things (IoT) applications, RTOS often serves as the control layer that coordinates sensing, communication, timing, and concurrency across multiple software tasks. As embedded devices increasingly incorporate wireless communication capabilities and application-specific logic, there is growing need for practical evaluation of how RTOS-level behavior interacts with communication protocols and application timing requirements.

Among available open-source RTOS platforms, Zephyr RTOS has become an important option for embedded development because of its modular architecture, broad board support, integrated build system, and support for common wireless stacks such as Bluetooth Low Energy (BLE). Zephyr is designed to run on resource-constrained microcontrollers while still providing kernel services such as threads, synchronization primitives, timing services, logging, and protocol integration. This makes it a strong candidate for research and prototype development involving concurrent embedded workloads.

Bluetooth Low Energy is especially relevant in modern embedded sensing systems because it enables low-power wireless communication between microcontroller-based nodes and mobile or gateway devices. In many small IoT deployments, BLE is used as the local wireless link for data transfer, while a smartphone or gateway forwards data to higher-level services such as Wi-Fi networks or cloud platforms. This gateway model is particularly useful for hardware platforms that support BLE but do not include integrated Wi-Fi.

This project investigates Zephyr RTOS on the Nordic nRF52840 Development Kit and uses that platform to build a wireless embedded sensing prototype. The work begins with development environment preparation and board validation, continues through RTOS threading and scheduling experiments, and then extends into BLE validation and custom application development. The final system is a custom BLE temperature-monitoring node that simulates sensor updates, exposes those values through a BLE GATT characteristic, and allows a mobile client to read live data. Experimental results are then used to characterize timing behavior, thread scheduling performance, and the effect of CPU interference on periodic task execution.

Rather than treating Zephyr only as a software framework to be configured, this project studies it as an experimental RTOS platform whose behavior can be seen and measured directly on real hardware. This makes the work relevant not only to embedded application development, but also to broader questions of scheduling behavior, wireless integration, and prototype-level performance characterization.

1.2 Problem Statement

Although RTOS platforms such as Zephyr provide extensive support for embedded systems development, successful use of these systems in practical wireless applications still requires more than simply building and flashing sample programs. Developers must first set up a stable development environment, verify correct board operation, understand kernel-level concepts such as threads and synchronization, and then integrate communication features such as BLE into a functioning application. Even after a working prototype is achieved, system-level performance behavior may remain insufficiently characterized.

Several challenges motivated this work.

First, development with Zephyr on embedded hardware can involve nontrivial setup issues, including toolchain configuration, board target selection, SDK integration, flashing support, and runtime debugging. These issues must be resolved before meaningful experimentation can begin.

Second, while Zephyr provides threading and scheduling abstractions, the practical behavior of those abstractions under different priority conditions is not immediately obvious from documentation alone. Questions such as how starvation occurs, how yielding differs from blocking, and how periodic tasks behave under interference are best answered through direct experimentation on hardware.

Third, BLE sample applications may compile and flash successfully yet still show runtime faults if memory or configuration settings are insufficient. This means wireless validation requires not only build success, but also runtime stability and client-side confirmation.

Fourth, building a working BLE sensing node is only part of the engineering problem. For the system to be useful in a research context, its timing behavior must also be measured and discussed quantitatively. Accordingly, this project includes evaluation of task latency and scheduler timing under both baseline and interference conditions.

Finally, while power consumption is an important embedded metric, obtaining trustworthy current measurements requires correct instrumentation and validated measurement setup. This project therefore also had to confront the methodological issue of distinguishing between attempted measurement and defensible measurement.

Taken together, these challenges define the central problem addressed in this project: how to design, implement, validate, and partially characterize a Zephyr RTOS-based BLE sensing node on the nRF52840 Development Kit in a way that is experimentally grounded, technically honest, and suitable for formal academic reporting.

1.3 Objectives

The primary goal of this project is to design and experimentally evaluate a Zephyr RTOS-based embedded BLE sensing node on the Nordic nRF52840 Development Kit.

The specific goals are:

1. To set up a complete Zephyr RTOS development environment capable of building and flashing applications to the nRF52840 DK.
2. To verify correct target-board operation through successful execution of a Zephyr sample application.
3. To study core Zephyr RTOS concepts, including threads, sleep-based timing, logging, synchronization, thread priorities, starvation, and fairness.
4. To confirm Bluetooth Low Energy functionality on the nRF52840 DK using a known Zephyr BLE peripheral example.
5. To build a custom BLE sensing application that simulates temperature data on the board.
6. To expose application data through a custom BLE GATT service and readable characteristic.
7. To verify end-to-end BLE interaction using nRF Connect on a mobile device.
8. To measure application-level task latency and scheduler timing behavior on the custom BLE node.
9. To compare periodic thread behavior under baseline and interference conditions.
10. To organize the collected data into tables, graphs, and report-ready experimental results.

1.4 Scope of Work

The scope of this project is limited to development and evaluation of an embedded wireless prototype based on Zephyr RTOS and the nRF52840 Development Kit.

The project includes:

- Zephyr environment setup and validation on a Windows host system
- building and flashing Zephyr applications to the nRF52840 DK
- RTOS experimentation involving threads, synchronization, and scheduler behavior.
- BLE validation using a stock Zephyr sample.
- development of a custom BLE node with simulated temperature updates

- BLE service and characteristic exposure to a mobile client
- timing-based experimental evaluation of the implemented system

The project does not aim to:

- full Wi-Fi implementation on the nRF52840 DK itself
- integration with a finished cloud backend
- physical temperature sensor hardware integration
- production-grade power measurement data
- a hard real-time or safety-critical guarantee of timing behavior

Because the nRF52840 DK does not provide onboard Wi-Fi for this project configuration, the intended architecture is best interpreted as a BLE node that can communicate with a phone or future gateway device, which may then provide onward connectivity to Wi-Fi or cloud infrastructure. In this sense, the work should be viewed as a BLE-based sensing and RTOS evaluation platform rather than a fully complete end-to-end IoT cloud product.

1.5 Contributions

The main contributions of this project are as follows:

1. A complete documented Zephyr RTOS bring-up workflow for the Nordic nRF52840 Development Kit on a Windows development host.
2. A practical RTOS study showing thread creation, periodic timing behavior, mutex-based synchronization, starvation under priority scheduling, the limits of yielding, and equal-priority fairness.
3. Successful validation of BLE functionality on the nRF52840 DK using a Zephyr peripheral sample, including diagnosis and correction of a runtime stack overflow issue through configuration changes.
4. Development of a custom BLE temperature-monitoring prototype that simulates sensor values, advertises as a connectable BLE node, and exposes live application data through a custom GATT characteristic.
5. Quantitative timing experiments measuring sensor-update-to-client-read latency and periodic scheduler behavior under both baseline and interference conditions.
6. A documented comparison showing that high-priority CPU interference imposed a large and repeatable delay on a lower-priority periodic task while preserving low jitter.

7. An honest methodological record of attempted but untrusted power measurement using the Nordic Power Profiler Kit II, preserving research transparency by excluding invalid current data from formal claims.

1.6 Organization of the Report

This report is organized as follows.

Chapter 2 presents the literature review together with an overview of Zephyr RTOS, Bluetooth Low Energy, and the embedded-system concepts most relevant to this project.

Chapter 3 describes the overall system design, including the hardware platform, software structure, RTOS threading model, BLE communication design, and sensing-node architecture.

Chapter 4 explains the implementation process across the completed project phases, including environment preparation, board validation, scheduler experimentation, BLE validation, and construction of the custom BLE temperature node.

Chapter 5 presents experimental results and discussion. This chapter includes scheduler observations, latency measurements, baseline and interference comparisons, graph-based summaries, and discussion of the attempted but incomplete power-measurement effort.

Chapter 6 concludes the report, summarizes the main findings, says the project limitations, and outlines directions for future work.

Chapter 2: Literature Review and Zephyr RTOS Overview

2.1 Introduction

This chapter presents the technical background needed for the research project. The study focuses on the development and experimental evaluation of a Zephyr RTOS-based BLE sensing node on the Nordic nRF52840 Development Kit. Because the system combines concurrent embedded execution, wireless communication, and timing-based performance analysis, several technical areas must be established before the system design, implementation, and results can be discussed in detail.

The first area is embedded real-time operating systems, which provide the scheduling, timing, and synchronization mechanisms required for structured execution on resource-constrained hardware. The second is Zephyr RTOS itself, which serves as both the implementation platform and the experimental environment used throughout this study. The third is Bluetooth Low Energy, which provides the wireless link between the embedded node and the mobile client. The fourth is embedded performance evaluation, particularly latency, periodic timing, timing error, jitter, and power consumption, since these metrics form the basis for assessing the behavior of the implemented system.

Together, these areas provide the conceptual foundation for the system architecture, implementation decisions, and experimental analysis presented in the chapters that follow.

2.2 Embedded Real-Time Operating Systems

An embedded real-time operating system is designed to support predictable and structured execution on resource-constrained hardware. In contrast to bare metal superloop designs, an RTOS allows a system to be organized into concurrent tasks or threads, each with defined priorities, timing behavior, and synchronization requirements. This makes RTOS platforms especially useful when an embedded application must coordinate sensing, communication, logging, and control activities in a disciplined manner.

In practical terms, RTOS provides several core services:

- task or thread management,
- timing services such as delays and sleep primitives,
- synchronization primitives such as mutexes and semaphores,
- interrupt handling,
- and scheduling policies that determine which execution context runs at a given time.

These services are important in systems where multiple activities must appear to progress together. For example, a sensing application may need to update data periodically, respond to a communication request, and log diagnostic output without collapsing into a single monolithic loop. RTOS allows these functions to be separated into concurrent execution units while still being coordinated under a single scheduling framework.

One of the most important elements of any RTOS is the scheduler. In priority-driven scheduling, higher-priority tasks generally preempt lower-priority tasks when they are ready to run. This supports rapid servicing of time-sensitive work, but it also introduces trade-offs involving starvation, fairness, blocking behavior, and response latency. The choice of scheduling policy therefore has direct implications for how reliably a periodic or communication-related task can be executed in the presence of competing workloads.

Another central RTOS concept is synchronization. When multiple tasks access the same resource without coordination, inconsistent behavior, corruption, or timing anomalies may result. For this reason, RTOS design commonly relies on synchronization primitives such as mutexes, semaphores, queues, and event flags to enforce controlled interaction between concurrent tasks.

Real-time systems are often classified as hard real-time, soft real-time, or best-effort real-time depending on how strict their timing guarantees must be. Hard real-time systems require deadlines to be met with very high assurance, while soft real-time systems tolerate occasional deadline misses but still require bounded and useful response behavior. Best-effort real-time systems aim for timely execution but do not claim strict deadline guarantees. In embedded wireless sensing applications, the practical classification depends on the application context, acceptable delay, and consequences of timing deviation.

Because this research project focuses on implementation and experimental characterization rather than formal schedulability analysis, its relevance to RTOS study lies in practical observation of concurrency, timing behavior, and priority effects on a real embedded target.

2.3 Zephyr RTOS

Zephyr RTOS is an open-source real-time operating system intended for embedded systems and resource-constrained devices. It supports multiple processor architectures and development boards, including ARM Cortex-M platforms such as the Nordic nRF52840 used in this project. Zephyr is widely used in embedded development because it combines kernel services, device-driver infrastructure, connectivity stacks, and a unified build system in a modular software framework.

One important aspect of Zephyr is its support for scalable kernel functionality. It provides threads, priorities, synchronization primitives, timers, work queues, and memory-management features that allow developers to build concurrent embedded applications with relatively clear

structure. These features make Zephyr useful not only for end-product development but also for research and teaching contexts in which scheduling behavior and system interaction need to be observed directly.

Another key aspect of Zephyr is its development workflow. Applications are typically managed using West, built with CMake and Ninja, and compiled through the Zephyr SDK or another supported toolchain. This unified workflow supports reproducible builds across supported boards and makes it easier to transition from sample applications to custom firmware.

Zephyr also includes integrated support for common embedded communication technologies, including Bluetooth Low Energy. This is especially useful for embedded IoT research because it allows the same platform to support both RTOS-level concurrency experiments and wireless application development.

From a research perspective, Zephyr is attractive because it occupies a useful middle ground. It is higher-level and more structured than bare-metal firmware development, yet still exposes the low-level timing, scheduling, and synchronization issues that are central to embedded systems. It therefore allows a system to be studied not only as software that runs, but also as a set of interacting real-time mechanisms whose behavior can be measured on hardware.

In addition, Zephyr is supported by extensive official documentation, board definitions, samples, and subsystem examples. This makes it well suited for incremental development, where a project may begin with known-good sample code and later extend into custom application logic.

For these reasons, Zephyr RTOS is an appropriate platform for a research project that seeks to combine embedded concurrency, BLE-based communication, and timing-oriented experimental evaluation on a real development board.

2.4 Bluetooth Low Energy in Embedded IoT Systems

Bluetooth Low Energy is a short-range wireless communication technology designed for devices that exchange relatively small amounts of data while operating under tight power constraints. It is widely used in embedded sensing systems, wearable devices, portable monitors, beacons, and mobile-connected peripherals. BLE is especially attractive when an embedded node must communicate directly with a smartphone or nearby gateway without the overhead of more power-intensive wireless technologies.

BLE communication is typically organized around the Generic Attribute Profile, or GATT. In this model, a peripheral device advertises its presence and exposes structured data through services and characteristics. A client device scans for nearby peripherals, connects to a selected device, and interacts with the available GATT entries through read, write, notify, or indicate

operations. This structure makes BLE well suited to sensing applications in which measured or generated data must be exposed in an organized and query-able form.

In embedded IoT systems, BLE often serves as the local communication layer rather than the full end-to-end networking solution. A common architecture is for the embedded node to communicate locally over BLE while a smartphone or gateway provides onward connectivity to Wi-Fi, the Internet, or cloud services. This separation is especially practical when the embedded board itself does not include direct support for a full wireless backhaul in the intended system configuration.

BLE is also relevant from a systems-performance standpoint. Even though BLE is often described as lightweight and low power, overall application behavior depends not only on the radio protocol but also on local scheduling, data-update timing, client access timing, and software design choices such as polling versus notifications. As a result, application-level responsiveness in a BLE sensing system is shaped jointly by the wireless stack and the surrounding embedded execution model.

In the context of this research area, BLE is therefore important for two reasons. First, it provides a realistic wireless mechanism for exposing embedded application data to an external client. Second, it introduces a communication pathway whose usefulness depends on how well it interacts with periodic tasks, shared data handling, and timing behavior inside the embedded node itself.

2.5 Embedded Performance Evaluation Metrics

Performance evaluation is an essential part of embedded systems research because a functioning prototype is not sufficient by itself. A useful embedded system must also be characterized in terms of how it behaves under execution. For RTOS-based wireless sensing systems, this characterization often includes timing behavior, responsiveness, scheduling consistency, and power consumption.

One important metric is **latency**. In embedded systems, latency generally refers to the time between an event of interest and the system's observable response to that event. Depending on the design, this may refer to interrupt response latency, task wake-up latency, communication latency, or application-level data-access latency. In wireless sensing applications, latency can also be interpreted in terms of data freshness, meaning how old the data is when it becomes visible to a client.

A second important metric is **periodic task timing**. Many embedded systems rely on tasks that are intended to execute at regular intervals. In practice, the actual observed period may differ from the nominal one because of scheduler overhead, contention, blocking behavior, interrupt activity, or communication workload. Measuring actual period is therefore important for understanding how closely the implemented system follows its intended execution rhythm.

A related concept is **timing error**, which describes the difference between the intended execution interval and the observed one. Closely associated with timing error is **jitter**, which describes cycle-to-cycle variation in task timing. These two measures are analytically useful because they distinguish between a task that is consistently delayed and a task whose execution time varies unpredictably from cycle to cycle.

Another important metric in embedded systems is **power consumption**. For battery-powered or portable devices, power use directly affects runtime, efficiency, and thermal behavior. In wireless embedded nodes, power consumption may vary significantly depending on whether the system is idle, advertising, connected, reading sensors, transmitting data, or running under heavier processing load. Accurate power characterization therefore requires careful instrumentation and trustworthy measurement conditions.

Finally, performance evaluation also includes how results are represented. Tables, graphs, and summarized statistics are not merely presentation choices; they are part of the analytical process because they turn raw logs and measurements into interpretable evidence. In an embedded research project, performance metrics become meaningful only when they are measured clearly, interpreted correctly, and communicated in a form that supports defensible conclusions.

2.6 Summary

This chapter has presented the technical background needed for the research project. Embedded real-time operating systems provide the concurrency, scheduling, timing, and synchronization mechanisms required for structured execution on resource-constrained hardware. Zephyr RTOS is an appropriate platform for such work because it combines kernel services, a unified build workflow, and integrated subsystem support for embedded development. Bluetooth Low Energy provides a practical wireless communication layer for exposing embedded data to nearby client devices. Performance metrics such as latency, periodic timing, timing error, jitter, and power consumption provide the basis for evaluating how an implemented system behaves in practice.

Together, these concepts establish the background needed to understand the system design and implementation presented in the following chapters.

Chapter 3: System Design

3.1 System Overview

The system developed in this research project is a Zephyr RTOS-based embedded wireless sensing node implemented on the Nordic nRF52840 Development Kit. The final prototype performs three core functions:

1. periodic generation of application-side temperature data,
2. exposure of that data through a custom BLE service and readable characteristic,
3. interaction with a mobile BLE client through nRF Connect.

At a high level, the system is not a fully cloud-connected IoT product. Instead, it is an experimentally useful embedded prototype that combines RTOS scheduling behavior with BLE-based wireless data access. This aligns closely with the project goals, which begin with Zephyr environment preparation and board validation, then move through Zephyr concepts, BLE example validation, construction of the research system, and performance measurement.

The final architecture that emerged from the completed work is simpler and more realistic than the original wording of “sensor thread, BLE thread, Wi-Fi thread.” Since the nRF52840 DK in this project configuration does not provide onboard Wi-Fi for the intended implementation path, the practical architecture became:

Sensor update on the nRF52840 DK → BLE exposure from the DK → smartphone BLE client → potential future gateway path to Wi-Fi/cloud

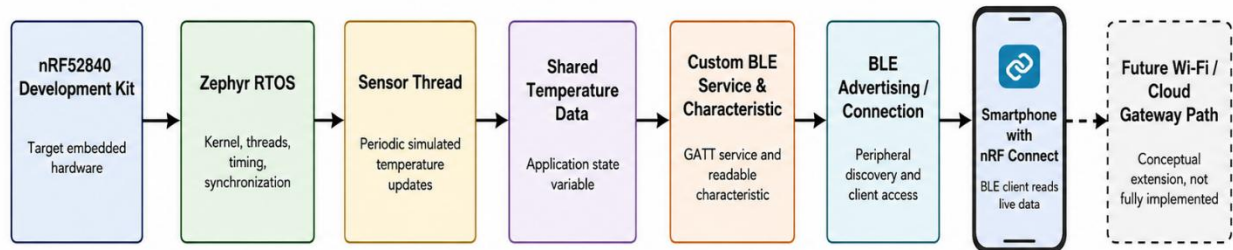
That interpretation is consistent with the goal document’s acknowledgement that BLE to smartphone to Wi-Fi to cloud is a common IoT pattern when Wi-Fi is not present directly on the embedded board.

The system also evolved in a logically staged way:

- **Phase 1** created the build, flash, and runtime foundation,
- **Phase 2** studied Zephyr threading and scheduler behavior,
- **Phase 3** validated BLE and implemented the custom node,
- **Phase 4** instrumented the node for timing experiments.

Because of that progression, the final system design is best understood not as a one-step design exercise, but as a research platform built incrementally and then characterized experimentally. Figure 3.1 shows the overall system architecture, including the nRF52840 DK, Zephyr RTOS, the sensor thread, the custom BLE service and characteristic, the nRF Connect mobile client, and the future smartphone-to-cloud gateway path.

Figure 3.1. Overall System Architecture



3.2 Development Platform and Hardware

The hardware platform for this research project is the **Nordic Semiconductor nRF52840 Development Kit**, used as the target board for all Zephyr builds and runtime experiments. Across all phases, the Zephyr board target used was:

nrf52840dk/nrf52840

The host development environment was a **Windows** system using **Windows PowerShell**. The Zephyr workspace was set up under:

C:\Users\feran\zephyrproject

and the Zephyr repository root used for builds was:

C:\Users\feran\zephyrproject\zephyr

The nRF52840 DK was selected because it provides:

- a supported Zephyr target,
- built-in BLE capability,
- easy flashing and debug access,
- compatibility with Nordic tools,

- and enough hardware support to allow real RTOS and wireless experiments on an actual embedded board rather than in simulation.

The board's role in the project was not simply to run a demo program. It served as:

- the execution platform for RTOS scheduling experiments,
- the BLE peripheral platform for wireless validation,
- and the target node for timing measurements.

From a hardware research perspective, this matters because all key results were obtained on real target hardware. The final report is therefore grounded in seen embedded execution rather than emulator-only or host-only behavior.

Host toolchain and support tools

The completed environment from Phase 1 consisted of:

- Python 3.12.7,
- West,
- CMake,
- Ninja,
- Zephyr SDK 0.16.5,
- Nordic *nrfjprog*,
- serial console access over COM10.

This toolchain became part of the practical system design because reproducible building and flashing were prerequisites for all later experimental work.

Mobile client platform

For wireless validation and characteristic reads, the mobile-side BLE client used throughout the project was **nRF Connect** on a smartphone. The phone served as the BLE client and future conceptual gateway in the broader system architecture. In Phase 3 and Phase 4, it was used to:

- scan for advertising devices,
- connect to the BLE peripheral,
- inspect services and characteristics,
- and read the custom temperature characteristic repeatedly.

3.3 Software Architecture

The software architecture of the final research system is centered on a custom Zephyr application developed outside the stock sample tree. The main custom application workspace used for the BLE node was:

C:\Users\feran\zephyrproject\phase3_goal5

The application followed the standard Zephyr project structure:

- *CMakeLists.txt*
- *prj.conf*
- *src/main.c*

The software evolved across phases, but by the time the final custom BLE node was set up, the architecture held the following functional components:

- **main initialization path**
- **sensor thread**
- **shared temperature state**
- **BLE stack initialization**
- **custom GATT service**
- **custom readable characteristic**
- **characteristic read callback**
- **timing instrumentation added during Phase 4**

This architecture was intentionally compact. The project was not trying to build a large production firmware stack; it was trying to build a small but experimentally meaningful embedded node whose behavior could be directly seen, changed, and measured.

Core execution structure

The final custom application used a thread-driven structure in which:

- the sensor thread updated a simulated temperature value periodically,
- BLE advertising exposed the device to the client,
- a read callback returned the current application value when the client requested it.

In Phase 4, the same structure was extended with timestamp capture so that:

- the time of the latest sensor update could be stored,

- the time of each client read could be compared against it,
- periodic execution intervals could be measured,
- and timing error could be calculated.

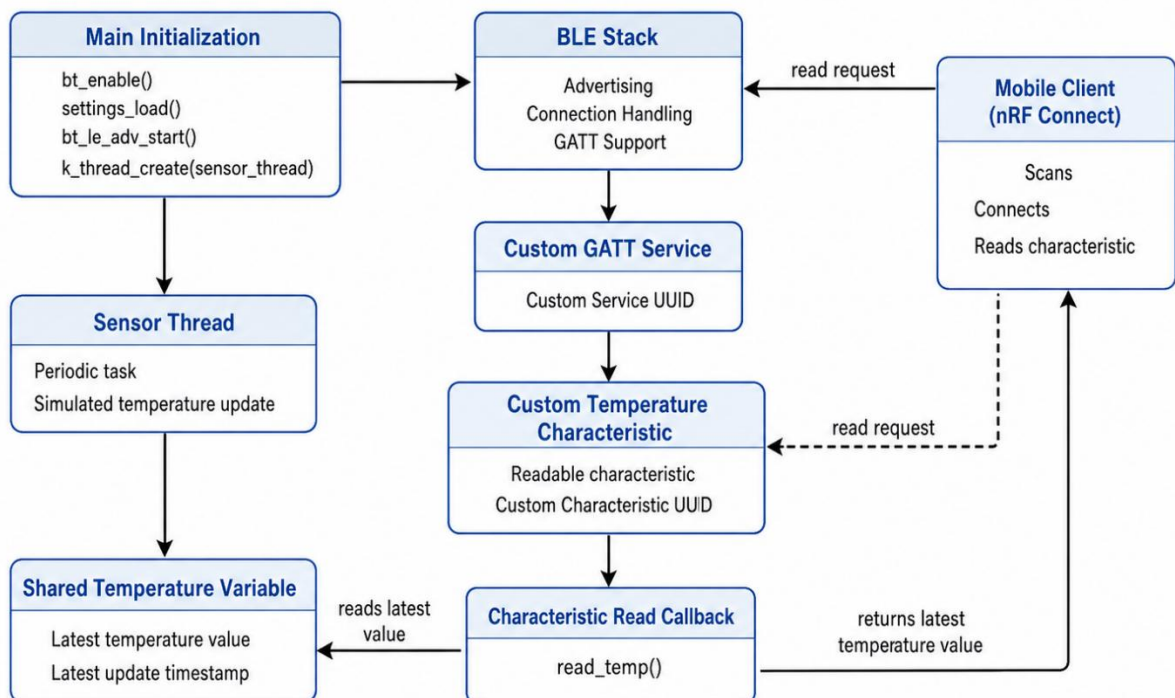
Design rationale

This modularity was important for research clarity. It allowed the project to separate:

- periodic application behavior,
- BLE communication behavior,
- and experimental timing instrumentation.

The final system is therefore simple enough to reason about directly but structured enough to support meaningful experiments. Figure 3.2 shows the software architecture of the custom Zephyr BLE temperature node, including main initialization, the sensor thread, shared application data, the BLE stack, the custom GATT service, the characteristic read callback, and client interaction.

Figure 3.2 Software Architecture of the Custom Zephyr BLE Temperature Node



The sensor thread periodically updates application data, while the BLE client accesses the latest value through the custom GATT characteristic and read callback.

3.4 Threading and Scheduling Design

One of the most important parts of the project design came from the Phase 2 RTOS experiments. Even though those experiments preceded the custom BLE node, they shaped how the final system was understood and instrumented.

The project goals explicitly required understanding Zephyr concepts such as:

- threads,
- timers/sleep,
- and logging,

and Phase 2 turned those concepts into actual scheduler experiments on the board.

Baseline threading model

The simplest execution pattern used in early Phase 2 included:

- main thread activity,
- Thread 1,
- Thread 2,
- periodic sleep intervals using `k_sleep()`.

This showed the core Zephyr model that multiple loops could be created and seen independently through serial output. It also showed that different sleep intervals produced the expected differences in observed output frequency.

Shared-resource synchronization

A design lesson from Phase 2 was that the UART console had to be treated as a shared resource. When multiple threads called `printf()` concurrently without protection, the output became visibly interleaved and corrupted. That problem was solved by introducing a mutex-protected print helper.

This became an important design principle for the project:

- concurrency exists even in simple embedded observation code,
- and shared resources must be synchronized if clean, interpretable behavior is needed.

Priority-driven scheduling

The next major design lesson was the effect of thread priorities. In Zephyr, a lower numeric value corresponds to a higher priority. In Phase 2, threads with different priorities were created and their behavior was seen under different blocking conditions. The main findings were:

- a continuously ready high-priority busy thread could starve a lower-priority periodic thread,
- *k_yield()* alone did not restore lower-priority progress in the tested case,
- a short blocking sleep in the high-priority thread allowed the lower-priority thread to run again,
- equal-priority threads with identical blocking patterns showed balanced behavior.

These results directly influenced the later Phase 4 experimental design, where timing was measured under both:

- baseline conditions,
- and interference from a high-priority busy thread.

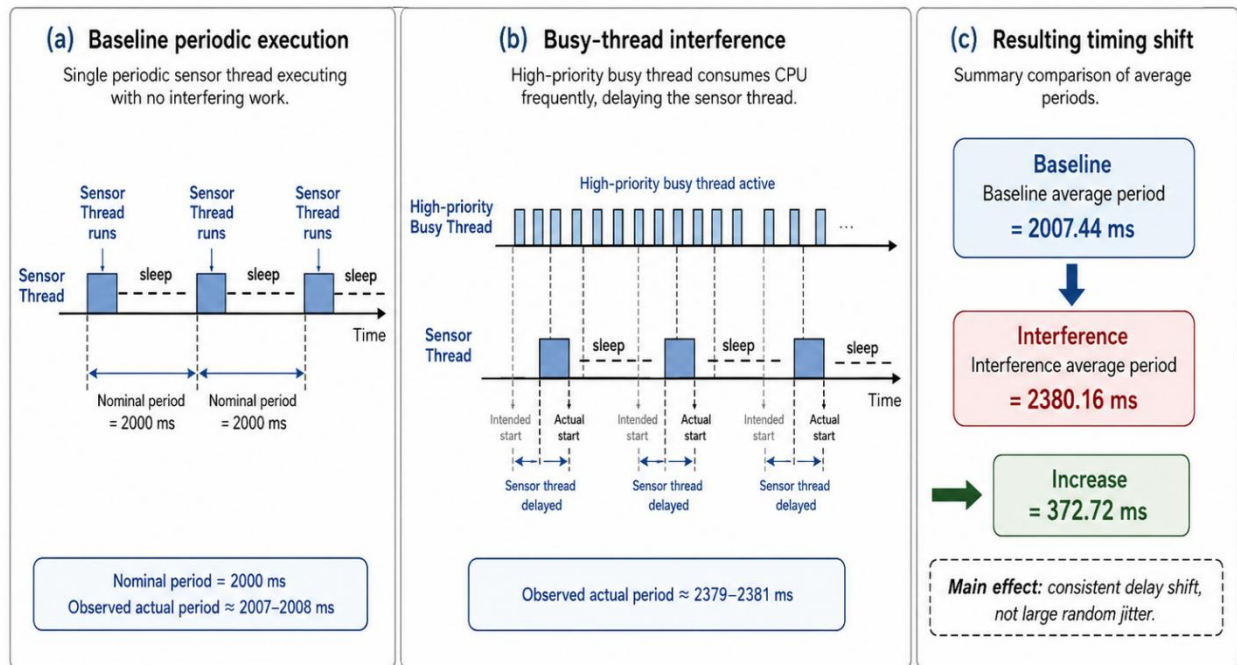
Final research-system threading interpretation

Although the final custom BLE node did not implement a large set of independent production threads, it did rely on the same RTOS principles:

- periodic thread execution,
- priority-aware scheduling,
- blocking behavior,
- and timing instrumentation.

The sensor thread in the final system became the main periodic workload used for later timing experiments. This design choice kept the final prototype small while still preserving a direct connection between RTOS behavior and measurable application timing. Figure 3.3 illustrates the thread interaction and scheduling concept under baseline and interference conditions.

Figure 3.3. Thread interaction and scheduling concept



Conceptual illustration of scheduler behavior for the periodic sensor thread under baseline execution and under interference from a higher-priority busy thread. Interference causes a consistent delay shift in the sensor thread period rather than highly irregular jitter.

3.5 BLE Communication Design

BLE communication formed the wireless core of the research system. The design was developed in two stages:

1. validation of the Zephyr BLE peripheral sample,
2. construction of the custom BLE temperature node.

Stock BLE validation stage

The first BLE design milestone was confirming the stock Zephyr BLE peripheral sample from:

samples\bluetooth\peripheral

This step was necessary because a custom BLE research application would have little value if the platform BLE path itself had not first been verified. During this phase, the BLE sample initially failed at runtime with an MPU fault and stack overflow during Bluetooth initialization. The problem was solved by applying an overlay configuration with increased stack and heap settings and enabling settings support.

That result had two design implications:

- BLE support on the board was real and usable,
- but runtime configuration mattered, not just compile success.

Custom BLE node stage

After BLE validation succeeded, the project moved to the custom application. The custom BLE node was designed to:

- advertise as a BLE peripheral,
- expose a custom primary service,
- expose a custom readable characteristic,
- return live application data to the client.

On the phone, the device appeared as **Phase3 Temp Node**, and the custom service UUID and characteristic UUID were visible in nRF Connect. Repeated characteristic reads returned changing values, proving that live application updates were reaching the client.

Advertising design

One notable practical design issue was that multiple advertising helper macros were not compatible with the exact Zephyr tree in use. The final reliable solution was to reuse the advertising startup pattern from the working stock sample and use:

```
BT_LE_ADV_CONN_FAST_1
```

inside the advertising start call.

This reflects an important embedded design principle: when APIs vary across versions, the most reliable approach is often to mirror the exact style used by a known-good sample in the same source tree.

Client interaction design

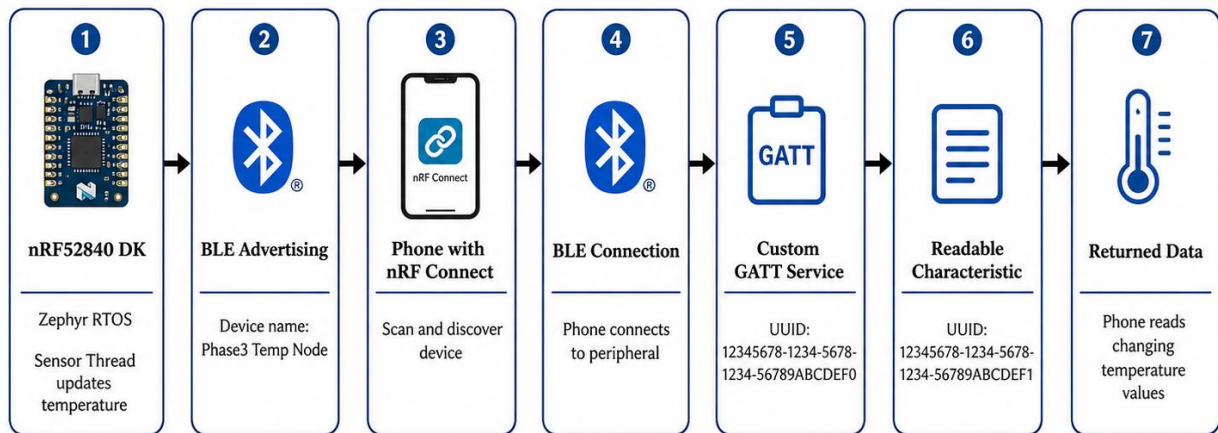
The BLE client interaction model was read-based rather than notify-based. The mobile client:

- connected to the node,
- browsed the custom service,
- manually read the characteristic.

That read-based model made the Phase 4 latency experiment possible, because each client read could be associated with the age of the latest sensor update at the time of access. Figure 3.4

illustrates the BLE communication flow from advertising and discovery through connection, service access, characteristic read, and returned temperature data.

Figure 3.4. BLE Communication Flow



BLE communication flow between the custom Zephyr temperature node and the mobile nRF Connect client.

3.6 Sensor Data Design

The sensing function in this project was implemented at the **simulated sensor** level rather than with a physical onboard temperature sensor. This was consistent with the goal document, which explicitly allowed simulation if no suitable physical sensor was used.

Simulated temperature generation

The custom application used a dedicated sensor thread to update a global temperature value periodically. The value changed over time, and the board-side serial output showed repeated updates such as:

- *Sensor thread updated temperature:* ... in Phase 3,
- later refined timestamped update logs in Phase 4.

This fulfilled the project’s need for application-side data generation while keeping the focus on RTOS behavior and BLE exposure rather than on physical analog sensor integration.

Shared data model

The current temperature value was stored in shared application state and served as the payload for the custom BLE characteristic. During Phase 4, this design was extended so that the system also stored:

- the time of the latest sensor update,
- the number of client reads,
- and later timing statistics derived from those values.

Why simulation was appropriate here

For this research project, the central research questions were:

- can Zephyr-based periodic application work be structured and measured,
- can live application data be exposed through BLE,
- how does scheduler behavior affect timing performance?

Those questions do not fundamentally require a physical temperature sensor. A simulated value that is updated through the same RTOS mechanisms is sufficient for studying:

- periodic task behavior,
- shared data exposure,
- BLE read timing,
- and interference effects.

Using a simulated sensor also reduced unnecessary variables and made the final experimental interpretation cleaner.

3.7 Experimental Design Strategy

The system was designed not only for functional operation, but also for experimental evaluation. This is an important distinction, because the final prototype was intended to serve as a measurable embedded research platform rather than only as a working BLE demonstration.

The experimental strategy focused on three primary performance categories:

- application-level read latency,

- periodic scheduler timing under baseline conditions,
- periodic scheduler timing under interference.

Latency measurement design

For latency evaluation, the design stored the timestamp of each sensor update and later compared it with the timestamp recorded during a BLE characteristic read. This made it possible to quantify the age of the most recent sensor value when the client accessed the characteristic. In this study, the resulting metric represents **sensor-update-to-client-read latency**, which is an application-level measure of data freshness.

Baseline scheduler timing design

For baseline timing evaluation, the periodic sensor thread was instrumented to record the time of each execution and the time elapsed since the previous cycle. The observed interval was then compared against the nominal 2000 ms task period. This made it possible to quantify both the actual period and the timing error under normal operating conditions.

Interference timing design

For interference testing, a higher-priority busy thread was introduced so that the same sensor-thread timing could be measured under CPU contention. This allowed direct comparison between unloaded and loaded cases and provided a practical way to evaluate how sustained higher-priority activity affected the execution of the lower-priority periodic task.

Power measurement attempt

The original experimental plan also included power-consumption measurement using the Nordic Power Profiler Kit II. That portion of the work was attempted, and supporting screenshots were collected, but the resulting values were not considered trustworthy enough for formal quantitative analysis. Accordingly, the power-consumption study is treated as an attempted but incomplete experiment rather than as a complete measurement result.

Data products

By the end of the experimental phase, the system had produced:

- raw serial timing logs,
- summarized timing tables,
- and graphs showing latency, average sensor-thread period, and average timing error under baseline and interference conditions.

This means the final system design succeeded not only in implementing a working BLE sensing node, but also in enabling structured collection of quantitative evidence for later analysis.

3.8 Summary

This chapter presents the overall system design of the research project. The final system is a Zephyr RTOS-based BLE sensing node implemented on the nRF52840 Development Kit and accessed through a smartphone BLE client. Its design combines a small custom application structure with RTOS-managed periodic sensing behavior, BLE service exposure, and later timing instrumentation for experimental evaluation.

The development platform, software structure, threading model, BLE communication path, and simulated sensor design were established through the staged completion of Phases 1 through 4. The result is a compact but experimentally meaningful embedded system that supports both functional BLE data access and quantitative timing analysis.

The next chapter describes how this design was implemented across the completed project phases.

Chapter 4: Implementation

4.1 Introduction

This chapter describes how the research system was implemented across the completed phases of the project. Although the final prototype appears relatively compact in its finished form, the implementation process was incremental and strongly dependent on the order in which problems were solved. The project did not begin with a fully working BLE sensing node. Instead, it progressed through four distinct phases:

- Phase 1: environment setup and board validation,
- Phase 2: Zephyr RTOS experimentation,
- Phase 3: BLE validation and custom BLE node implementation,
- Phase 4: timing-performance measurement and data preparation.

That phased progression matters because each later implementation step depended on the correctness of the earlier ones. The custom BLE node in Phase 3 would not have been meaningful without the development environment from Phase 1, the scheduler understanding from Phase 2, and the timing instrumentation introduced in Phase 4. As a result, this chapter is organized not as a flat list of code files, but as a practical implementation narrative that shows how the final system was established.

4.2 Development Environment Setup

The first implementation stage was to set up a functioning Zephyr RTOS development environment on a Windows host system for the nRF52840 DK. This work was completed in Phase 1 and created the foundation for everything that followed.

4.2.1 Host environment and workspace

The main workspace used in the project was:

C:\Users\feran\zephyrproject

and the Zephyr repository root used for builds was:

C:\Users\feran\zephyrproject\zephyr

The final working environment included:

- Windows PowerShell,
- Python 3.12.7,
- West,

- CMake,
- Ninja,
- Zephyr SDK 0.16.5,
- Nordic *nrfjprog*,
- serial access through COM10.

4.2.2 Python compatibility resolution

A major early implementation difficulty was Python interpreter compatibility. Multiple Python versions were present on the host, including a newer version that was too new for some of the dependency wheels used during Zephyr setup. This created mismatches between:

- the Python interpreter invoked by *python*,
- the *pip* installation target,
- and the interpreter detected by CMake during Zephyr build.

The issue was eventually resolved by settling on Python 3.12.7, which matched the smallest requirement reported by the active Zephyr version. Once the correct Python interpreter was consistently selected, dependency installation and build configuration began succeeding reliably.

4.2.3 Board target correction

Another important implementation issue involved the correct board target syntax. Older Zephyr naming conventions initially led to unsuccessful build attempts, but the correct target for the active environment was eventually identified as:

nrf52840dk/nrf52840

This was not a trivial naming detail. It directly affected whether Zephyr could resolve the board and configure a valid build.

4.2.4 SDK and toolchain integration

The Zephyr SDK was set up at:

C:\zephyr-sdk

and detected correctly by Zephyr once the required environment pathing and support tools were resolved. The final successful environment reported both host tools and toolchain detection for Zephyr SDK 0.16.5.

4.2.5 Build and flash pipeline verification

Once the environment stabilized, the *hello_world* sample was successfully built and linked, with Zephyr producing the expected firmware artifacts. Nordic flashing support was then confirmed using:

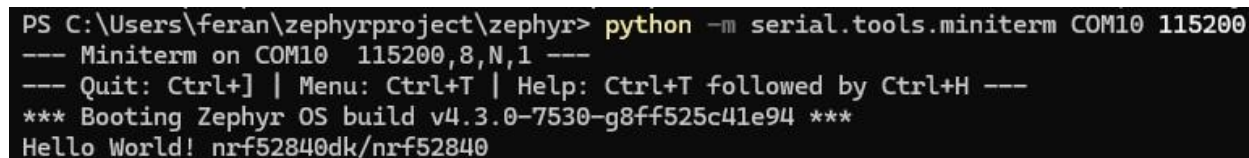
```
west flash --runner nrfjprog
```

instead of the default runner path that had initially failed. Finally, the serial console confirmed successful target execution with the Zephyr boot banner and *Hello World!* output.

From an implementation perspective, the most important result of Phase 1 was not simply that a sample built once, but that a reproducible workflow now existed:

- build,
- flash,
- reset,
- observe runtime output,
- iterate.

That repeatable workflow made later code experimentation practical. Figure 4.1 shows the serial console output proving successful Zephyr boot and execution of the *hello_world* sample on the nRF52840 DK.



```
PS C:\Users\feran\zephyrproject\zephyr> python -m serial.tools.miniterm COM10 115200
--- Miniterm on COM10 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
*** Booting Zephyr OS build v4.3.0-7530-g8ff525c41e94 ***
Hello World! nrf52840dk/nrf52840
```

Figure 4.1: Phase 1 successful Hello World serial output screenshot.

4.3 Board Validation and Initial Zephyr Application Work

After the environment was operational, the board itself had to be confirmed as a reliable execution platform. Phase 1 completed the first board validation through *hello_world*, but the first meaningful custom implementation work began in Phase 2 with a dedicated custom application directory:

```
C:\Users\feran\zephyrproject\phase2_goal3
```

This application used the standard Zephyr structure:

- *CMakeLists.txt*

- *prj.conf*
- *src/main.c*

The first custom implementation was intentionally simple. It created:

- one main execution context,
- two worker threads,
- repeated *printk()* output,
- and periodic delays through *k_sleep()*.

The purpose of this first custom application was not to build the final research node yet. Its purpose was to verify that the board could reliably execute a user-authored Zephyr multithreaded application and that thread behavior could be seen through the serial console.

This proved important because it set up two things:

1. the project was no longer dependent on stock samples only,
2. the board was already functioning as an RTOS research platform.

4.4 Zephyr Threading and Synchronization Implementation

Phase 2 provided the first substantial custom implementation of RTOS behavior on the nRF52840 DK. These experiments were not just conceptual; they shaped the later design and measurement logic used in the final custom node.

4.4.1 Multithreaded execution

The first multithreaded implementation created two Zephyr threads with different sleep intervals and allowed them to run alongside the main loop. This produced the expected qualitative execution pattern:

- the thread with the shortest sleep interval printed most often,
- the thread with the longer sleep interval printed less often,
- and the main loop appeared least often.

This simple implementation sets up a working basis for later scheduler experiments.

4.4.2 Shared UART output and mutex protection

Once multiple threads were printing concurrently, the UART output became visibly interleaved and corrupted. This was the first concrete implementation problem caused by concurrency:

- multiple threads were accessing a shared console path without synchronization.

The solution was to introduce a Zephyr mutex and wrap all console printing inside a protected helper function. This implementation change restored clean serial output and became one of the most practically important RTOS lessons of the project.

4.4.3 Priority experiments and starvation

The next implementation stage introduced explicit priorities, including a high-priority busy thread and a lower-priority periodic thread. When the high-priority thread ran continuously without sleeping, the lower-priority thread effectively disappeared from the observed output, showing starvation.

A later attempt inserted `k_yield()` into the high-priority busy thread, but this did not restore lower-priority progress. Only when the high-priority busy thread inserted a short blocking sleep did the lower-priority periodic thread resume visible execution.

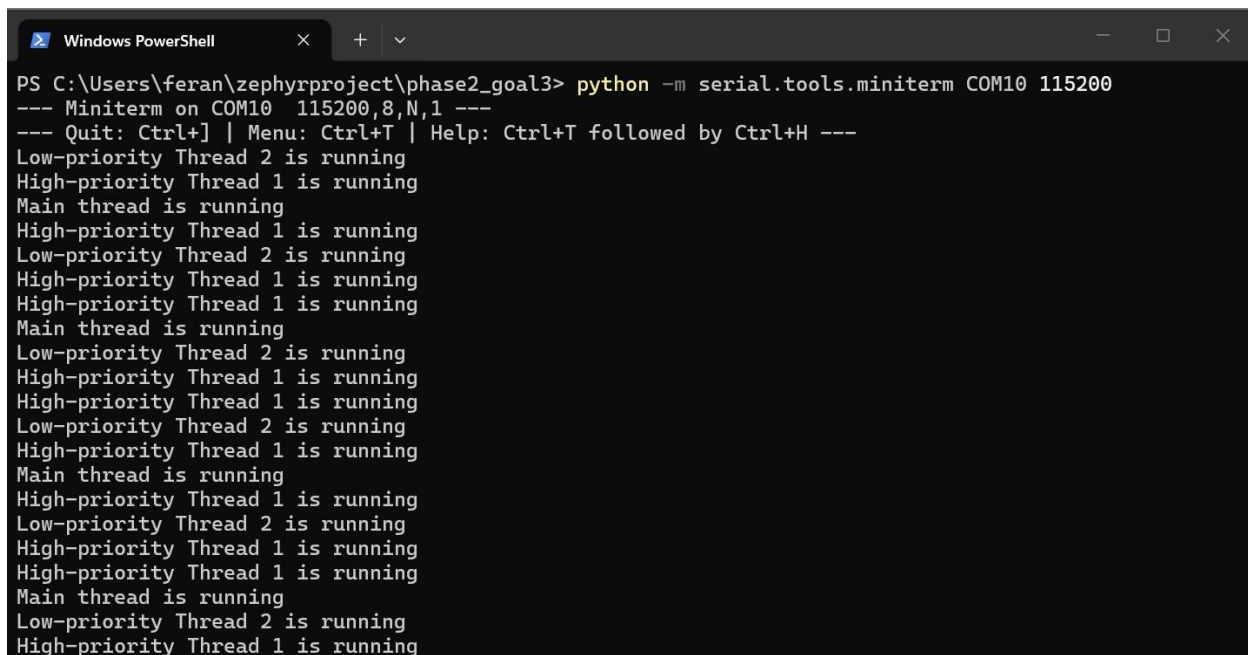
From an implementation perspective, this sequence mattered because it showed that:

- voluntarily yielding while staying ready was not enough in the tested case,
- actual blocking was necessary to create a true scheduling opportunity for the lower-priority work.

4.4.4 Equal-priority case

The last major Phase 2 implementation case created two equal-priority threads with identical periodic sleep behavior. The result was balanced execution between the two worker threads.

This provided an important contrast with the starvation case and later helped motivate the more quantitative Phase 4 timing experiments. Figure 4.2 shows a screenshot of serial terminal showing clean mutex-protected multithreaded output from main thread, high-priority thread, and low-priority thread.



```
PS C:\Users\feran\zephyrproject\phase2_goal3> python -m serial.tools.miniterm COM10 115200
--- Miniterm on COM10 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
Low-priority Thread 2 is running
High-priority Thread 1 is running
Main thread is running
High-priority Thread 1 is running
Low-priority Thread 2 is running
High-priority Thread 1 is running
High-priority Thread 1 is running
Main thread is running
Low-priority Thread 2 is running
High-priority Thread 1 is running
High-priority Thread 1 is running
Low-priority Thread 2 is running
High-priority Thread 1 is running
Main thread is running
High-priority Thread 1 is running
Low-priority Thread 2 is running
High-priority Thread 1 is running
High-priority Thread 1 is running
Main thread is running
Low-priority Thread 2 is running
High-priority Thread 1 is running
```

Figure 4.2: Screenshot of mutex-protected clean multithreaded output.

4.5 BLE Validation Implementation

BLE implementation began in Phase 3. This phase first used the Zephyr peripheral sample as a known working target before trying the custom application.

4.5.1 Stock BLE sample build path

The BLE validation sample used was:

samples\bluetooth\peripheral

and it was built in a separate dedicated build directory to avoid conflict with the earlier *hello_world* build directory.

This was an important implementation detail because Zephyr build directories are tied to source applications. Reusing a build directory across unrelated applications created configuration conflicts that had to be resolved by using a pristine separate build directory.

4.5.2 BLE runtime failure and overlay fix

Although the BLE sample eventually built and flashed successfully, the first runtime execution failed with an MPU fault and stack overflow during Bluetooth initialization. This proved that BLE implementation on the board required attention not only to build success, but also to runtime resource configuration.

To fix the problem, an overlay configuration file was created. The main changes included:

- larger main stack,
- larger system workqueue stack,
- larger heap memory pool,
- *CONFIG_SETTINGS=y*,
- *CONFIG_BT_SETTINGS=y*.

Once that overlay was merged correctly into the build, the BLE sample initialized successfully and began advertising.

4.5.3 Mobile validation

The final BLE validation step used nRF Connect on a mobile device to confirm:

- discovery of the advertising device,
- successful connection,
- visibility of services.

This completed the stock BLE implementation path and proved that the board could function as a functioning BLE peripheral under Zephyr. Figure 4.4 shows successful BLE sample runtime output proving Bluetooth initialization and advertising start on the nRF52840 DK and figure 4.5 shows nRF Connect view proving successful connection to the Zephyr BLE peripheral sample (“Zephyr Peripheral Sample Long”).

```

Windows PowerShell
[Progress] 0.000s | Programming image - block 1 of 1 [Progress] 0.000s | Programming image - Write successful [Progress] 0.000s | Check image validity - Initialize device info [Progress] 0.000s | Check image validity - block 1 of 2
[Progress] 1.232s | Program file - Done programming [Progress] 0.000s | Verify file - Check image [Progress] 2.261s | Check image validity - block 1 of 2
[Progress] 0.000s | Verify file - Done verifying [Progress] 0.000s | Verify file - Verifying [Progress] 0.000s | Verifying image - Verify successful
[Progress] 0.010s | Check image validity - Finished [Progress] 0.000s | Verifying image - block 1 of 1
[Progress] 1.273s | Verify file - Done verifying
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 736.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 629.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 911.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3087.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3112.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3349.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3160.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3176.
Applying system reset.
Run.
--- runners.nrfjprog: Board(s) with serial number(s) 1050230858 flashed successfully.
PS C:\Users\feran\zephyrproject\zephyr> python -m serial.tools.miniterm COM10 115200
--- Miniterm on COM10 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
*** Booting Zephyr OS build v4.3.0-7530-g8ff525c41e9d ***
[00:00:00.258,850] %[0m<inf> fs_nvs: 8 Sectors of 4096 bytes%[0m
[00:00:00.258,850] %[0m<inf> fs_nvs: alloc wra: 0, fh8%[0m
[00:00:00.258,850] %[0m<inf> fs_nvs: data wra: 0, 58%[0m
[00:00:00.261,169] %[0m<inf> bt_hci_core: HW Platform: Nordic Semiconductor (0x0002)%[0m
[00:00:00.261,199] %[0m<inf> bt_hci_core: HW Variant: nRF52x (0x0002)%[0m
[00:00:00.261,230] %[0m<inf> bt_hci_core: Firmware: Standard Bluetooth controller (0x00) Version 4.3 Build 99%[0m
[00:00:00.261,657] %[0m<inf> bt_hci_core: No ID address. App must call settings_load()%[0m
Bluetooth initialized
[00:00:00.262,176] %[0m<inf> bt_hci_core: HCI transport: Controller%[0m
[00:00:00.262,268] %[0m<inf> bt_hci_core: Identity: D7:0B:AE:D8:78:95 (random)%[0m
[00:00:00.262,298] %[0m<inf> bt_hci_core: HCI: version 5.4 (0x0d) revision 0x0000, manufacturer 0x05f1%[0m
[00:00:00.262,359] %[0m<inf> bt_hci_core: LMP: version 5.4 (0x0d) subver 0xffff%[0m
Advertising successfully started
Indicate VND attr 0x28d34 (UUID 12345678-1234-5678-1234-56789abcdef1)

```

Figure 4.4: Successful BLE sample serial output screenshot showing Bluetooth initialized and advertising started.

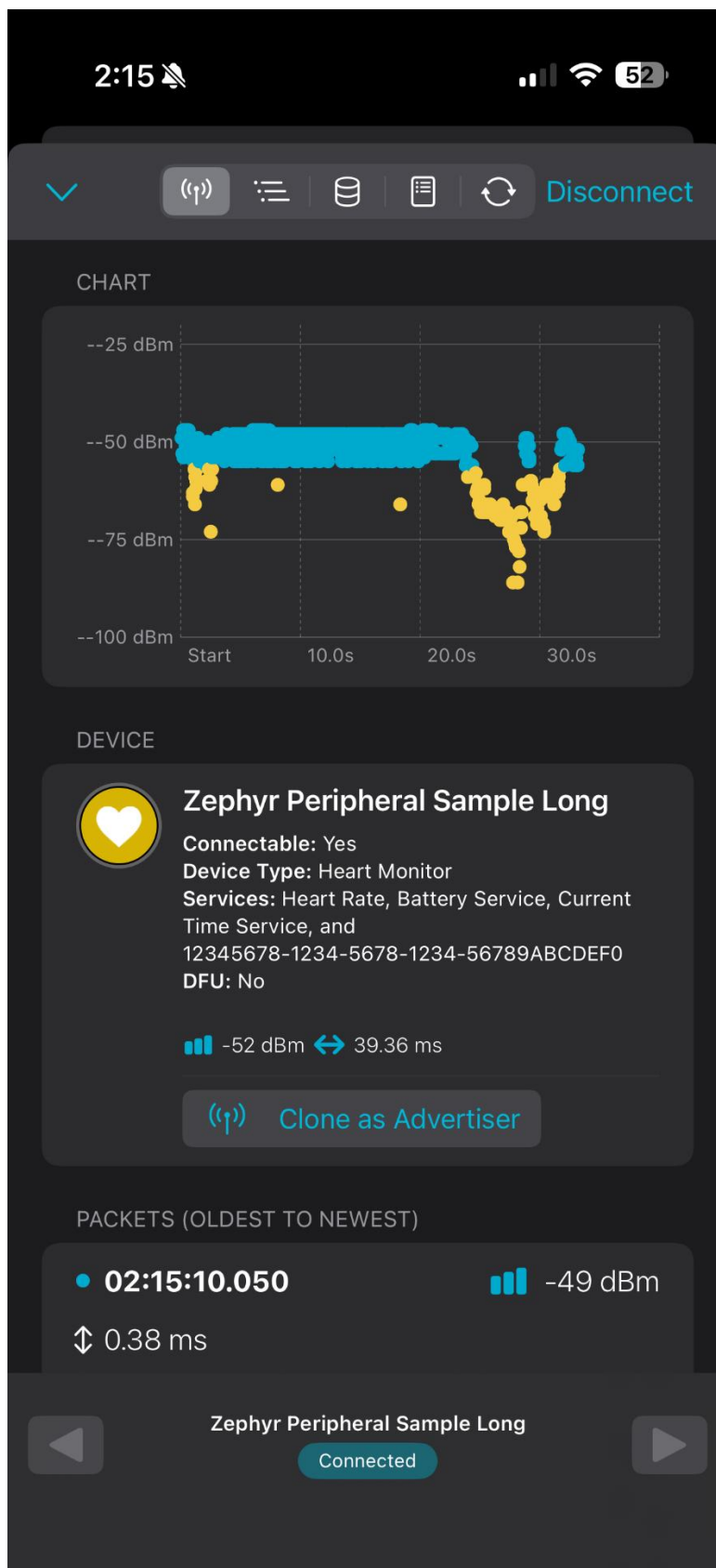


Figure 4.5: nRF Connect screenshot showing Zephyr Peripheral Sample Long connected.

4.6 Custom BLE Temperature Node Implementation

After BLE functionality was confirmed, the research system moved to implementation of the custom BLE node in the directory:

C:\Users\feran\zephyrproject\phase3_goal5

4.6.1 Project files

The custom application used:

- *CMakeLists.txt*
- *prj.conf*
- *src/main.c*

The project configuration included:

- BLE support,
- peripheral role support,
- device naming,
- settings support,
- stack and heap sizing,
- serial/console support for debugging and logging.

4.6.2 Initial custom application structure

The custom node implemented:

- a globally simulated temperature value,
- a sensor thread,
- a custom service,
- a custom readable characteristic,
- BLE advertising.

The sensor thread updated the temperature periodically using simulated values. This satisfied the project's sensor-integration goal at the simulated level, which was explicitly allowed by the goals document if a real sensor was not used.

4.6.3 Compile-time corrections

Several implementation issues had to be corrected before the custom app became stable:

- missing declaration of *settings_load()*,
- advertising macro mismatches across Zephyr API variants,
- incompatibility of some helper macros with the exact Zephyr version in use.

The most reliable fix came from inspecting the working stock sample and reusing its advertising call style. Specifically, the final custom BLE advertising used the same helper style as the local sample:

- *BT_LE_ADV_CONN_FAST_1*

This was a practical implementation lesson in version-aware embedded development.

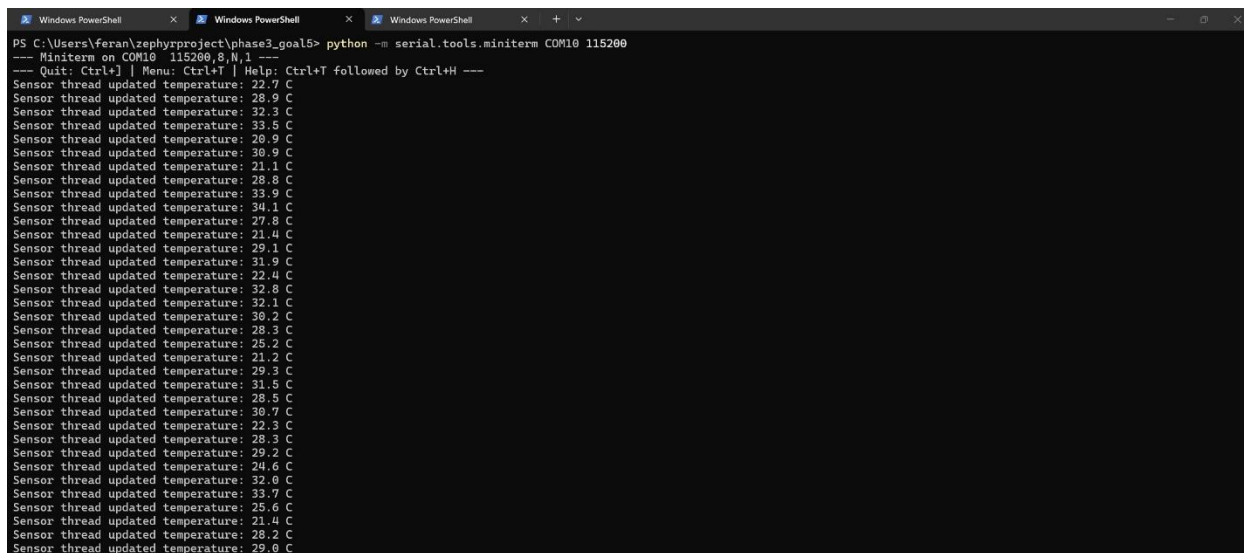
4.6.4 Mobile-side verification of custom node

Once the custom app built and flashed successfully, it advertised as:

Phase3 Temp Node

The phone discovered it, connected successfully, and displayed the custom service and characteristic. Repeated reads of the custom characteristic returned changing values, proving that the live application state on the board was being exposed correctly to the BLE client.

This was the first full implementation of the actual research system. Figure 4.6 shows serial console output demonstrating periodic temperature updates from the custom sensor thread on the nRF52840 DK, figure 4.7 shows nRF Connect view demonstrating successful connection to the custom BLE device “Phase3 Temp Node” and figure 4.8 shows nRF Connect service view demonstrating the custom GATT service and readable characteristic implemented by the Phase3 Temp Node.



```

PS C:\Users\feran\zephyrproject\phase3_goal5> python -m serial.tools.miniterm COM10 115200
--- Miniterm on COM10 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
Sensor thread updated temperature: 22.7 C
Sensor thread updated temperature: 28.9 C
Sensor thread updated temperature: 32.3 C
Sensor thread updated temperature: 33.5 C
Sensor thread updated temperature: 28.9 C
Sensor thread updated temperature: 30.9 C
Sensor thread updated temperature: 21.1 C
Sensor thread updated temperature: 28.8 C
Sensor thread updated temperature: 33.9 C
Sensor thread updated temperature: 34.1 C
Sensor thread updated temperature: 27.8 C
Sensor thread updated temperature: 21.4 C
Sensor thread updated temperature: 29.1 C
Sensor thread updated temperature: 31.9 C
Sensor thread updated temperature: 22.4 C
Sensor thread updated temperature: 32.8 C
Sensor thread updated temperature: 32.1 C
Sensor thread updated temperature: 30.2 C
Sensor thread updated temperature: 28.3 C
Sensor thread updated temperature: 25.2 C
Sensor thread updated temperature: 21.2 C
Sensor thread updated temperature: 29.3 C
Sensor thread updated temperature: 31.5 C
Sensor thread updated temperature: 28.5 C
Sensor thread updated temperature: 30.7 C
Sensor thread updated temperature: 22.3 C
Sensor thread updated temperature: 28.3 C
Sensor thread updated temperature: 29.2 C
Sensor thread updated temperature: 24.6 C
Sensor thread updated temperature: 32.6 C
Sensor thread updated temperature: 33.7 C
Sensor thread updated temperature: 25.6 C
Sensor thread updated temperature: 21.4 C
Sensor thread updated temperature: 28.2 C
Sensor thread updated temperature: 29.0 C

```

Figure 4.6: Serial screenshot showing sensor thread temperature updates.

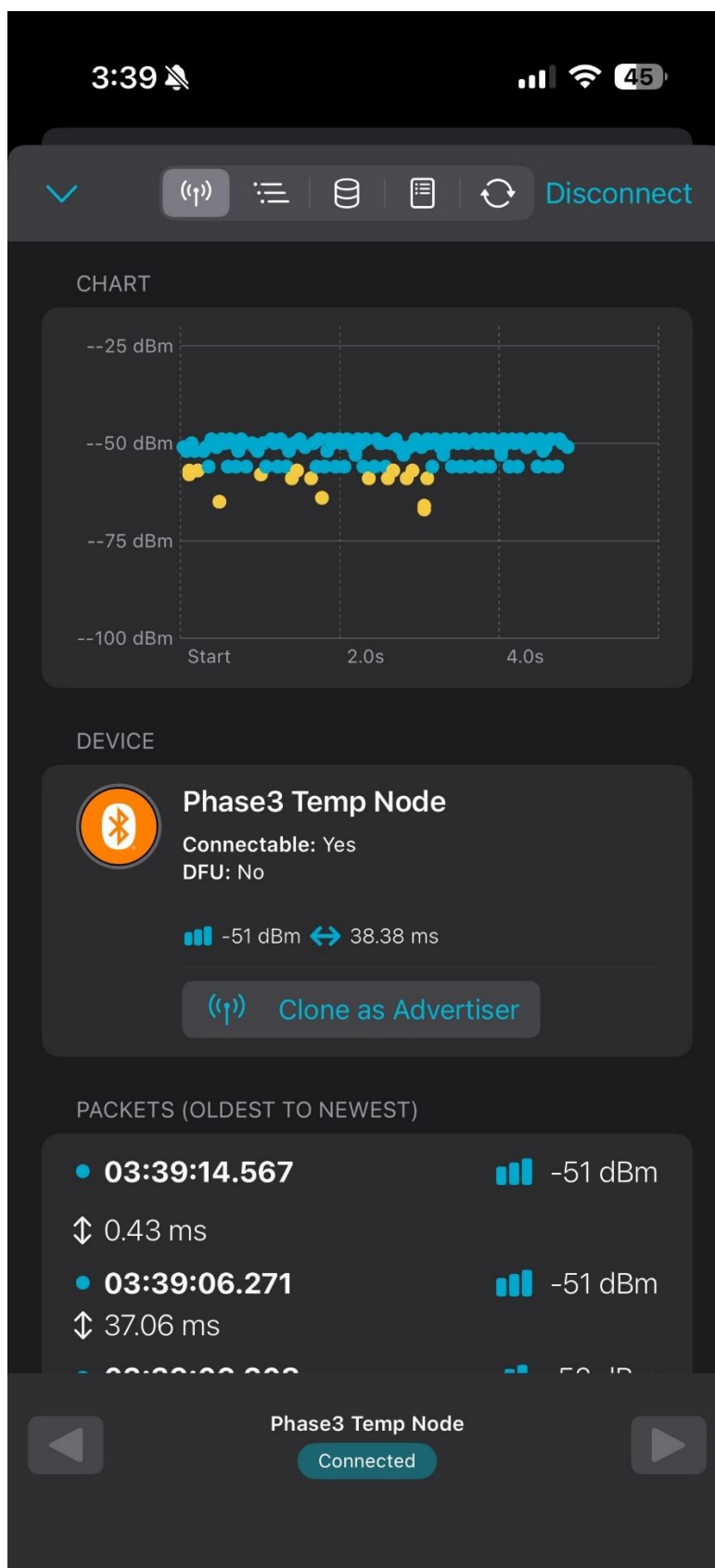


Figure 4.7: nRF Connect screenshot showing Phase3 Temp Node connected.

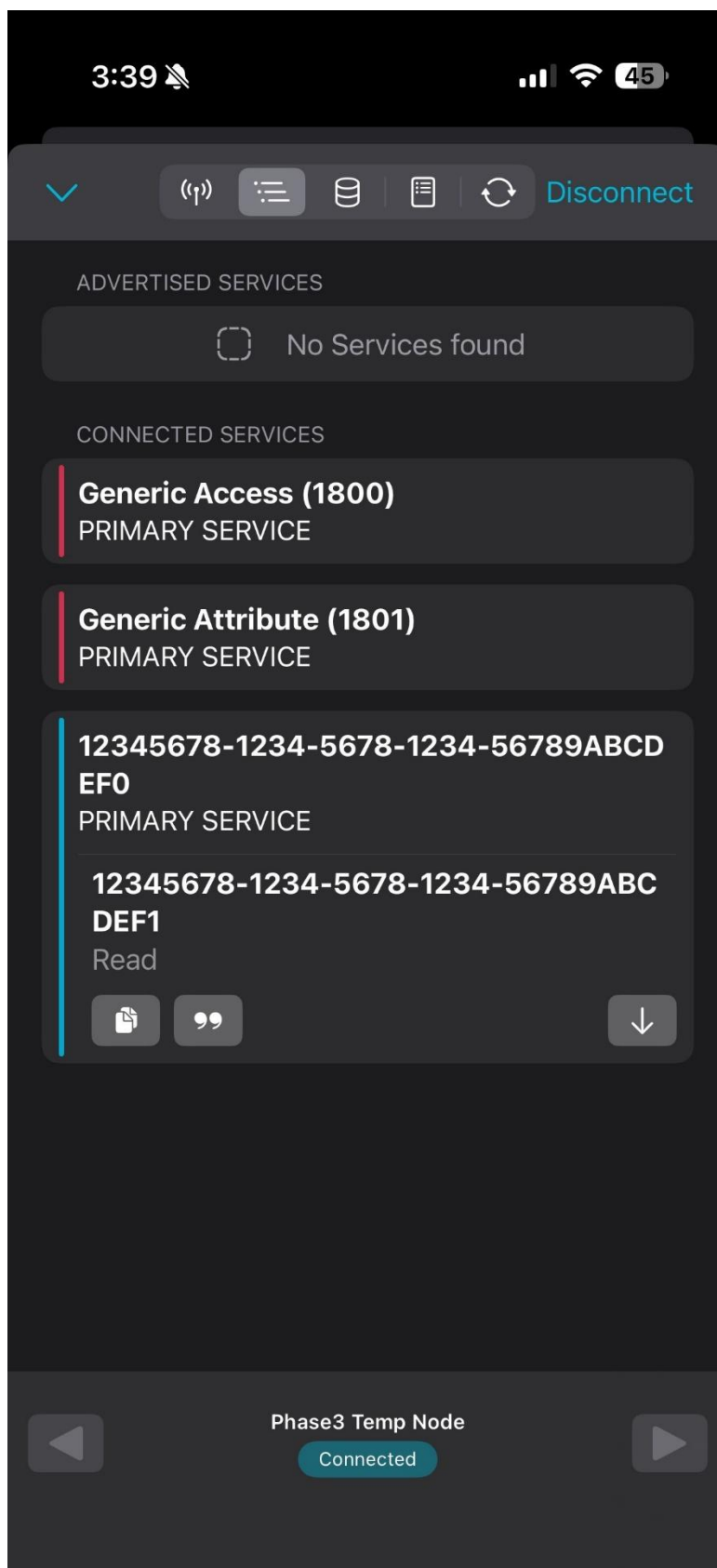


Figure 4.8: nRF Connect screenshot showing custom service and characteristic.

4.7 Timing Instrumentation Implementation

Phase 4 did not redesign the custom BLE node from scratch. Instead, it instrumented the existing node so that timing performance could be measured directly.

4.7.1 Latency instrumentation

For task-latency measurement, the implementation added:

- a variable to store the timestamp of the latest sensor update,
- logic inside the characteristic read callback to capture the current time,
- a computed difference between read time and update time,
- logging of each BLE read number, returned temperature, and computed latency.

This allowed the system to report application-level latency directly through serial output whenever the BLE client read the characteristic.

4.7.2 Baseline scheduler timing instrumentation

For the baseline timing experiment, the sensor thread was changed to record:

- the current time at each cycle,
- the earlier update time,
- the actual observed period,
- the error compared to the intended 2000 ms interval.

This transformed the sensor thread from a simple periodic update loop into a measurable periodic task.

4.7.3 Interference implementation

For the interference experiment, a high-priority busy thread was added back into the application alongside the periodic sensor thread. The busy thread continuously burned CPU time and only slept briefly after certain print intervals.

This was an intentional implementation choice designed to create controlled CPU contention and make the scheduler impact measurable.

4.8 Data Logging and Graph Preparation

The final part of implementation involved organizing experiment outputs into report-ready forms.

Serial output was captured to log files using PowerShell redirection and Tee-Object, allowing the raw timing lines to be preserved. From these logs, summary statistics were calculated and later plotted in graphs for:

- Phase 4A task latency,
- average sensor-thread period baseline vs interference,
- average sensor-thread timing error baseline vs interference.

This stage was part implementation and part data-preparation workflow because it transformed the system into something experimentally communicable rather than merely executable.

4.9 Attempted Power Measurement Implementation

The final implementation step tried in Phase 4 was current measurement using the Nordic Power Profiler Kit II. The intention was to support Goal 8's power-consumption requirement.

However, even though:

- the board could still run code,
- the profiler could display traces,
- and screenshots were captured,

the values obtained were implausibly small and not credible for the actual active firmware state being measured. A later hardware-side attempt to reconfigure measurement paths also made the setup unreliable enough that the power experiment was closed without trusted data.

From an implementation standpoint, this section is still important because it shows that:

- the project did try the measurement,
- the tooling was introduced,
- but the resulting data was excluded from formal results for methodological reasons.

That decision should be preserved in the final report exactly because it reflects correct research practice.

4.10 Design Decisions and Trade-Offs

Several implementation decisions shaped the final system:

Simulated sensing rather than physical sensor integration

This reduced hardware uncertainty and kept the focus on RTOS and BLE behavior. Since the goals explicitly allowed simulation, this was a valid trade-off.

Read-based BLE exposure rather than notification-based streaming

This made the client interaction simple and directly enabled the Phase 4 latency experiment.

Direct reuse of working sample idioms

Instead of guessing API behavior across Zephyr versions, the implementation used the local stock BLE sample as the compatibility reference. This was a practical and reliable choice.

Priority-based interference rather than synthetic timing math alone

Instead of only reasoning abstractly about scheduler effects, the implementation created an actual high-priority competing thread and measured its effect on the periodic task.

Excluding invalid current data

Although it would have been tempting to force the power-measurement attempt into the results section, the project instead chose the more defensible research path: document the attempt, keep the screenshot as evidence, and avoid making unsupported quantitative claims.

4.11 Summary

This chapter described how the research system was implemented from environment setup through the final timing experiments. The implementation proceeded incrementally:

- Phase 1 showed the Zephyr toolchain and verified board operation,
- Phase 2 implemented custom multithreaded Zephyr applications and scheduler experiments,
- Phase 3 implemented BLE validation and the custom BLE temperature node,
- Phase 4 instrumented that node for timing experiments and tried, but did not complete, power profiling.

The final implemented system was therefore more than a simple embedded BLE demo. It was a working Zephyr RTOS research platform whose behavior could be seen, changed, and measured directly on hardware.

The next chapter presents the experimental results and discussion derived from this implementation.

Chapter 5: Experimental Results and Discussion

5.1 Experimental Setup

The experimental evaluation in this research project was designed to quantify the behavior of the custom Zephyr RTOS-based BLE sensing node after functional implementation had already been completed. By the start of Phase 4, the system had already shown:

- a working Zephyr environment,
- verified board operation on the nRF52840 DK,
- successful BLE validation using a stock Zephyr sample,
- and a functioning custom BLE node with a simulated temperature source and readable BLE characteristic.

This meant the focus of experimentation could shift from “does it work?” to “how does it behave under measurement?”

The experimental work in this chapter is drawn primarily from Phase 2 and Phase 4:

- **Phase 2** supplied scheduler-behavior observations and timestamped qualitative experiments involving multithreading, starvation, blocking, yielding, and equal-priority behavior.
- **Phase 4** supplied the main quantitative timing dataset, including task latency, baseline periodic timing, and timing under interference, together with tables and graphs prepared for the final report.

Hardware and runtime platform

All experiments were conducted on the **Nordic nRF52840 Development Kit** running Zephyr RTOS, with build and flash operations performed from a Windows host environment. The board target used throughout the project was:

```
nrf52840dk/nrf52840
```

Runtime observation and log capture were performed through the serial console on **COM10**, while BLE interaction was conducted using **nRF Connect** on a smartphone.

Experimental categories

The final experimental work can be grouped into four categories:

1. **Scheduler behavior experiments from Phase 2**
2. **Task latency measurement from Phase 4A**

3. Scheduler timing measurement under baseline conditions from Phase 4B

4. Scheduler timing measurement under interference from Phase 4C

A fifth category, power measurement using the Nordic Power Profiler Kit II, was tried in Phase 4D but is presented separately as an incomplete experiment because the values obtained were not considered trustworthy enough for formal analysis.

Measurement philosophy

An important methodological point is that the project did not try to overclaim what was measured. For example:

- the latency experiment measured **sensor-update-to-client-read latency**, not pure low-level BLE packet transmission delay,
- and the power experiment was documented as attempted but excluded from formal quantitative results because the measurement setup did not yield credible values.

That distinction matters because it keeps the results section technically honest.

5.2 Thread Scheduling Experiments from Phase 2

Before the custom BLE node was instrumented for quantitative timing measurements, the project first investigated Zephyr scheduler behavior qualitatively in Phase 2. These experiments were important because they set up the scheduler concepts later quantified in Phase 4.

5.2.1 Basic periodic thread execution

The first custom Zephyr experiments created multiple threads with different `k_sleep()` intervals. The observed serial output showed the expected pattern:

- the 1-second thread printed most often,
- the 2-second thread printed less often,
- the 3-second main loop printed least often.

This showed correct multithreaded execution and confirmed that blocking sleep intervals translated into observable differences in runtime frequency.

5.2.2 Shared UART output and synchronization

When multiple threads wrote to the UART console concurrently without synchronization, the output became garbled and interleaved. This provided direct evidence that the console path behaved as a shared resource and required protection. A Zephyr mutex was then introduced around the print path, and the serial output became clean and readable.

This result was important because it linked RTOS theory to visible hardware behavior. It also justified later disciplined logging practices in the measured experiments.

5.2.3 Starvation under a non-blocking high-priority thread

A later Phase 2 experiment introduced a CPU-intensive high-priority busy thread alongside a low-priority periodic thread. When the high-priority thread did not block, the lower-priority thread effectively disappeared from the observed output. The log showed repeated high-priority activity and main-thread activity, but no visible output from the lower-priority thread.

This was a clear demonstration of starvation.

5.2.4 Yielding versus blocking

The next experiment inserted *k_yield()* into the busy high-priority thread. Contrary to naive expectations, this did not restore lower-priority progress in the tested case. The high-priority thread remained ready and continued to dominate scheduler choice. Only when a short blocking sleep was added to the high-priority thread did the lower-priority periodic thread resume observable execution.

This distinction between:

- staying ready after yielding,
- and blocking with sleep,

became one of the most important conceptual findings of the Phase 2 scheduler study.

5.2.5 Equal-priority fairness

The final scheduler experiment in Phase 2 created two worker threads with equal priority and identical blocking behavior. The resulting timestamped output showed regular paired execution from both threads, without starvation of either.

This provided a useful contrast with the unequal-priority starvation case and showed that balanced execution could be obtained when priorities and sleep behavior were symmetrical.

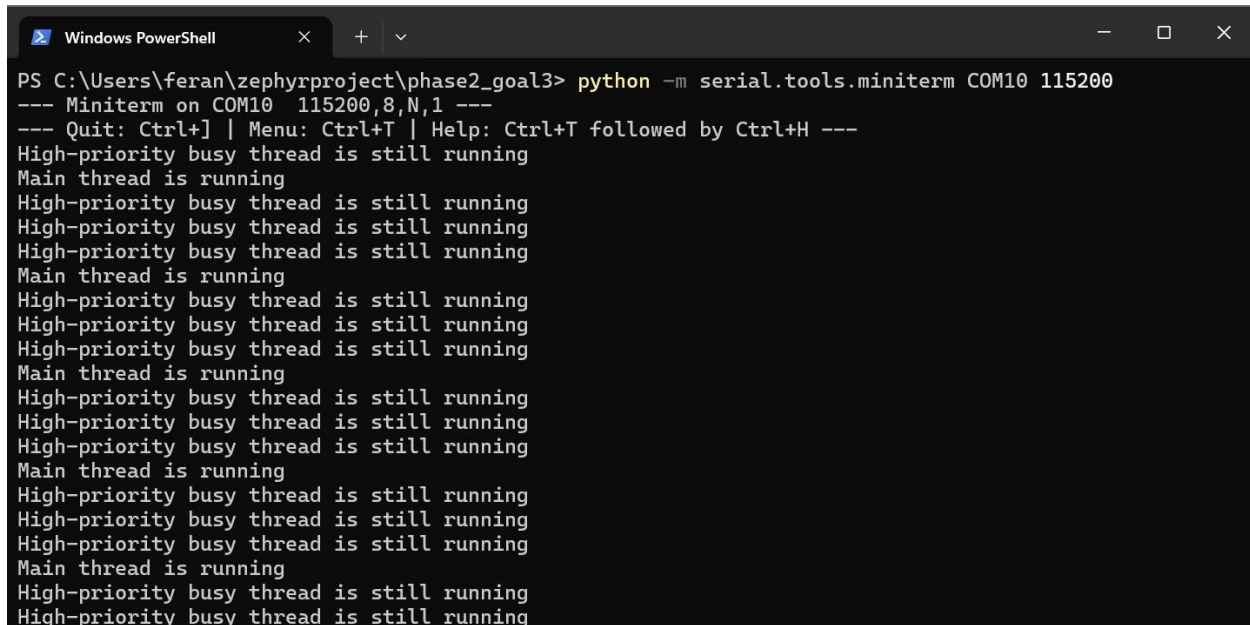
Discussion of Phase 2 scheduler experiments

Although Phase 2 was more qualitative than Phase 4, it was still an important part of the results record. It showed:

- correct thread creation,
- timing behavior with *k_sleep()*,
- the necessity of synchronization,
- starvation under continuous high-priority CPU demand,

- the ineffectiveness of yielding alone in the tested priority case,
- and balanced execution for equal-priority threads.

These findings motivated the later quantitative Phase 4 timing experiments. Figure 5.1 shows representative serial log excerpts from Phase 2 scheduler experiments: (a) starvation of the low-priority thread under a continuously ready high-priority busy thread, (b) restoration of low-priority execution after insertion of a short blocking sleep in the high-priority thread, and (c) balanced execution of two equal-priority threads with identical periodic sleep behavior.

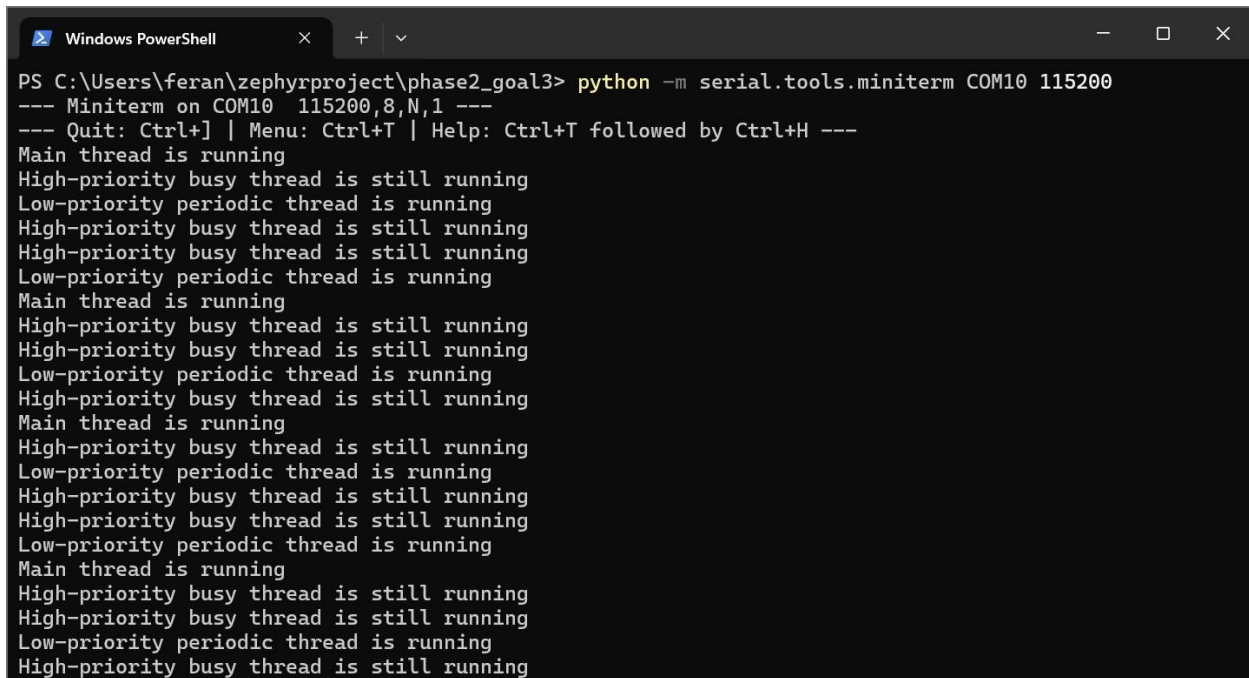


```

PS C:\Users\feran\zephyrproject\phase2_goal3> python -m serial.tools.miniterm COM10 115200
--- Miniterm on COM10 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
High-priority busy thread is still running
Main thread is running
High-priority busy thread is still running
High-priority busy thread is still running
High-priority busy thread is still running
Main thread is running
High-priority busy thread is still running
High-priority busy thread is still running
High-priority busy thread is still running
Main thread is running
High-priority busy thread is still running
High-priority busy thread is still running
High-priority busy thread is still running
Main thread is running
High-priority busy thread is still running
High-priority busy thread is still running
High-priority busy thread is still running
Main thread is running
High-priority busy thread is still running
High-priority busy thread is still running

```

Figure 5.1(a): Starvation case

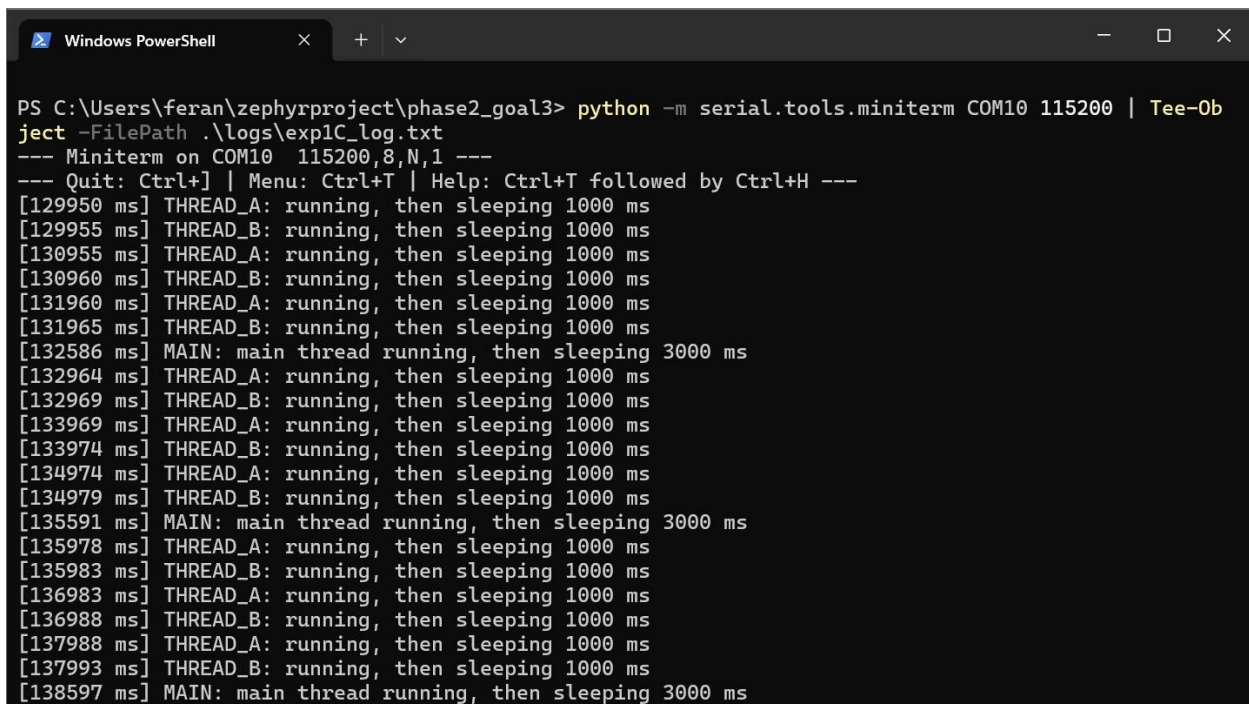


```

PS C:\Users\feran\zephyrproject\phase2_goal3> python -m serial.tools.miniterm COM10 115200
--- Miniterm on COM10 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
Main thread is running
High-priority busy thread is still running
Low-priority periodic thread is running
High-priority busy thread is still running
High-priority busy thread is still running
Low-priority periodic thread is running
Main thread is running
High-priority busy thread is still running
High-priority busy thread is still running
Low-priority periodic thread is running
High-priority busy thread is still running
Main thread is running
High-priority busy thread is still running
Low-priority periodic thread is running
High-priority busy thread is still running
High-priority busy thread is still running
Low-priority periodic thread is running
Main thread is running
High-priority busy thread is still running
High-priority busy thread is still running
Low-priority periodic thread is running
High-priority busy thread is still running

```

Figure 5.1(b): Restoration case



```

PS C:\Users\feran\zephyrproject\phase2_goal3> python -m serial.tools.miniterm COM10 115200 | Tee-Object -FilePath .\logs\exp1C_log.txt
--- Miniterm on COM10 115200,8,N,1 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
[129950 ms] THREAD_A: running, then sleeping 1000 ms
[129955 ms] THREAD_B: running, then sleeping 1000 ms
[130955 ms] THREAD_A: running, then sleeping 1000 ms
[130960 ms] THREAD_B: running, then sleeping 1000 ms
[131960 ms] THREAD_A: running, then sleeping 1000 ms
[131965 ms] THREAD_B: running, then sleeping 1000 ms
[132586 ms] MAIN: main thread running, then sleeping 3000 ms
[132964 ms] THREAD_A: running, then sleeping 1000 ms
[132969 ms] THREAD_B: running, then sleeping 1000 ms
[133969 ms] THREAD_A: running, then sleeping 1000 ms
[133974 ms] THREAD_B: running, then sleeping 1000 ms
[134974 ms] THREAD_A: running, then sleeping 1000 ms
[134979 ms] THREAD_B: running, then sleeping 1000 ms
[135591 ms] MAIN: main thread running, then sleeping 3000 ms
[135978 ms] THREAD_A: running, then sleeping 1000 ms
[135983 ms] THREAD_B: running, then sleeping 1000 ms
[136983 ms] THREAD_A: running, then sleeping 1000 ms
[136988 ms] THREAD_B: running, then sleeping 1000 ms
[137988 ms] THREAD_A: running, then sleeping 1000 ms
[137993 ms] THREAD_B: running, then sleeping 1000 ms
[138597 ms] MAIN: main thread running, then sleeping 3000 ms

```

Figure 5.1(c): Equal-priority fairness case

5.3 Task Latency Evaluation

The first quantitative timing experiment in Phase 4 measured application-level task latency for the custom BLE sensing node. Specifically, the system measured the time between:

- the most recent sensor update on the board,
- and the moment the BLE characteristic was read by the mobile client.

5.3.1 Measurement method

To perform this measurement, the custom application was instrumented so that:

- the sensor thread recorded the timestamp of its latest update,
- the BLE characteristic read callback captured the current time at each read,
- and the difference between the two was printed on the serial log.

This produced a software-level measure of how “fresh” the returned data was when the client accessed the characteristic.

It is important to interpret this correctly: the metric is not pure radio transmission latency. It is **sensor-update-to-client-read latency**, or equivalently, the age of the most recent sensor value at the time of access.

5.3.2 Recorded latency data

Ten BLE reads were recorded. The measured latencies were:

- 1783 ms
- 1151 ms
- 1122 ms
- 1927 ms
- 946 ms
- 1453 ms
- 1142 ms
- 616 ms
- 1193 ms
- 1902 ms

These values are summarized below.

Table 5.1: Task Latency Measurements

Read #	Latency (ms)
1	1783
2	1151
3	1122
4	1927
5	946
6	1453
7	1142
8	616
9	1193
10	1902

5.3.3 Statistical summary

From the 10 measured reads:

- **Minimum latency:** 616 ms
- **Maximum latency:** 1927 ms
- **Average latency:** 1323.5 ms
- **Median latency:** 1172 ms

Table 5.2: Task Latency Summary

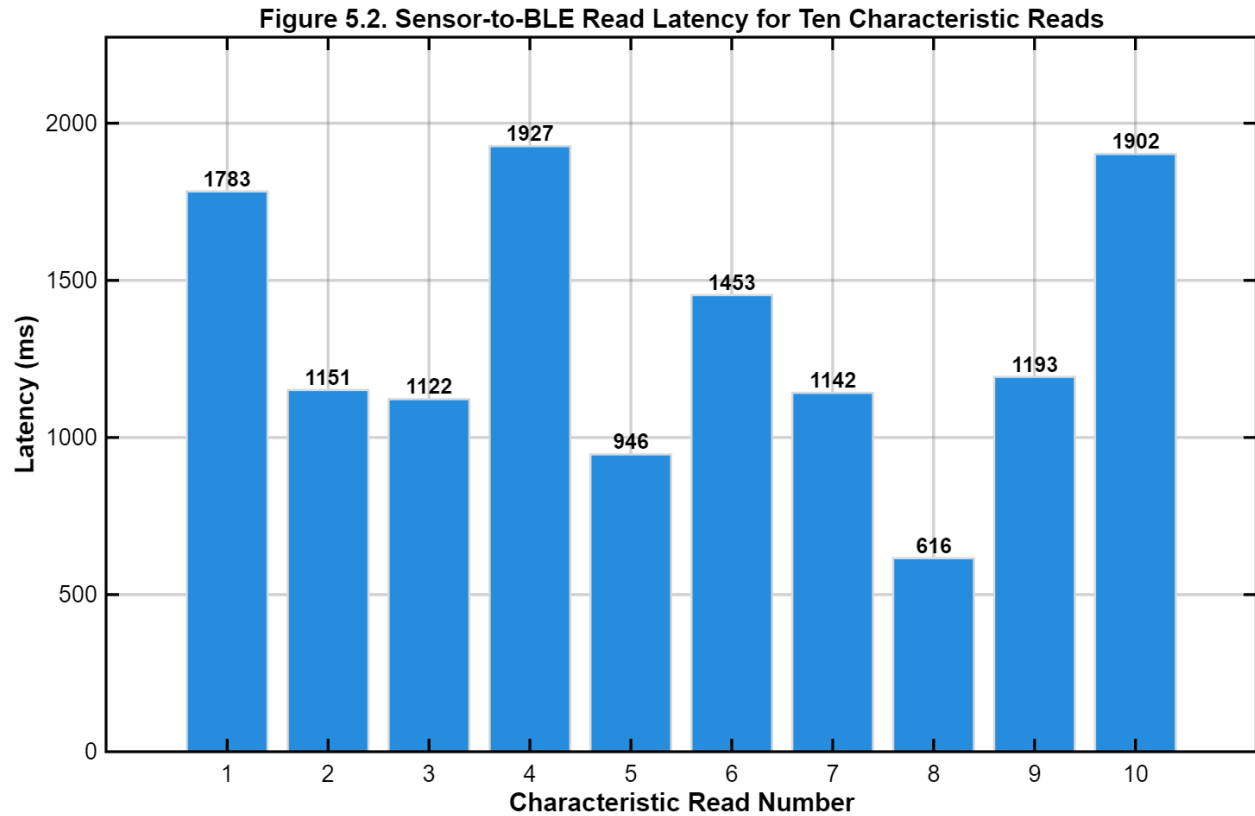
Metric	Value
Samples	10
Minimum	616 ms
Maximum	1927 ms
Average	1323.5 ms
Median	1172 ms

5.3.4 Interpretation

The measured latency varied substantially from one read to another. This is expected given the structure of the experiment. The sensor thread updated periodically every 2 seconds, while the phone read the characteristic asynchronously. As a result, the client sometimes read shortly after a sensor update and sometimes read much later in the cycle.

Therefore, the latency values should be interpreted as a **data freshness metric** rather than as a low-level BLE transmission-time metric.

The important result is that the custom BLE node successfully exposed live data whose age at the time of access could be measured and characterized. Figure 5.2 is the task latency graph that corresponds to the 10 measured latency values.



5.4 Scheduler Timing Under Baseline Conditions

The second quantitative experiment in Phase 4 measured the timing stability of the periodic sensor thread under normal, unloaded conditions. The intended thread period was 2000 ms. The experiment measured the actual time between consecutive sensor-thread executions and computed the error compared to the nominal period.

5.4.1 Measurement method

The sensor thread was instrumented to log:

- the cycle number,
- the current update timestamp,
- the actual period since the earlier update,
- and the timing error compared to 2000 ms.

5.4.2 Recorded baseline behavior

A total of **48 cycles** were recorded. Across those cycles:

- actual period remained between **2007 ms** and **2008 ms**

- timing error remained between +7 ms and +8 ms

5.4.3 Baseline summary

Table 5.3: Baseline Scheduler Timing Summary

Metric	Baseline Value
Samples	48
Min period	2007 ms
Max period	2008 ms
Avg period	2007.44 ms
Min error	+7 ms
Max error	+8 ms
Avg error	+7.44 ms
Jitter range	1 ms

5.4.4 Interpretation

This was an extraordinarily strong baseline result. Under light load:

- the periodic sensor thread ran extremely consistently,
- the actual period was only slightly above the nominal 2000 ms target,
- the cycle-to-cycle variation was exceedingly small.

In practical terms, the system showed:

- low overhead,
- low jitter,
- and highly repeatable timing in the unloaded case.

This result provided a clean reference point for comparison with the interference case.

5.5 Scheduler Timing Under Interference

The third quantitative experiment repeated the sensor-thread timing measurement while introducing a higher-priority busy thread to create CPU contention. This test was designed to show how the periodic task's timing changed when a continuously active higher-priority workload competed for execution time.

5.5.1 Measurement method

The same sensor-thread instrumentation from the baseline case was kept, but an added high-priority busy thread was introduced. This busy thread continuously consumed CPU time and slept only briefly at intervals, creating sustained interference.

5.5.2 Recorded interference behavior

A total of **45 cycles** were recorded. Under interference:

- actual period shifted to between **2379 ms** and **2381 ms**
- timing error shifted to between **+379 ms** and **+381 ms**

5.5.3 Interference summary

Table 5.4: Interference Scheduler Timing Summary

Metric	Interference Value
Samples	45
Min period	2379 ms
Max period	2381 ms
Avg period	2380.16 ms
Min error	+379 ms
Max error	+381 ms
Avg error	+380.16 ms
Jitter range	2 ms

5.5.4 Interpretation

The interference did not produce highly erratic timing. Instead, it produced a **large and consistent delay**.

That distinction is important:

- the periodic sensor task was still stable,
- but it was stably late.

This means the higher-priority busy thread imposed a significant scheduling penalty on the lower-priority periodic task, yet the resulting delay remained highly repeatable.

5.6 Baseline vs Interference Comparison

The strongest timing result in the project comes from direct comparison of the baseline and interference cases.

5.6.1 Comparison table

Table 5.5: Baseline vs Interference Scheduler Comparison

Metric	Baseline	Interference	Difference
Samples	48	45	—
Avg period	2007.44 ms	2380.16 ms	+372.72 ms
Min period	2007 ms	2379 ms	+372 ms
Max period	2008 ms	2381 ms	+373 ms
Avg error	+7.44 ms	+380.16 ms	+372.72 ms
Jitter range	1 ms	2 ms	+1 ms

5.6.2 Interpretation

Compared with the unloaded case, the busy high-priority thread increased the average measured period of the sensor thread by:

$$2380.16 \text{ ms} - 2007.44 \text{ ms} = 372.72 \text{ ms}$$

This is a significant shift.

However, the jitter remained low in both cases:

- 1 ms baseline,
- 2 ms under interference.

So, the main effect of interference was not randomness. It was a **stable, repeatable delay**.

This is one of the most important conclusions of the timing study, because it shows that under the tested Zephyr scheduling conditions:

- contention from a higher-priority CPU-intensive thread can substantially shift the execution time of a lower-priority periodic task,
- but the shift may remain consistent enough to characterize quantitatively.

That is a much more precise result than simply saying that “performance got worse under load.” Figure 5.3 is a graph comparing the average sensor thread period under baseline and interference, and figure 5.4 is a graph comparing the average timing error under baseline and interference.

Figure 5.3. Average Sensor Thread Period Under Baseline and Interference Conditions

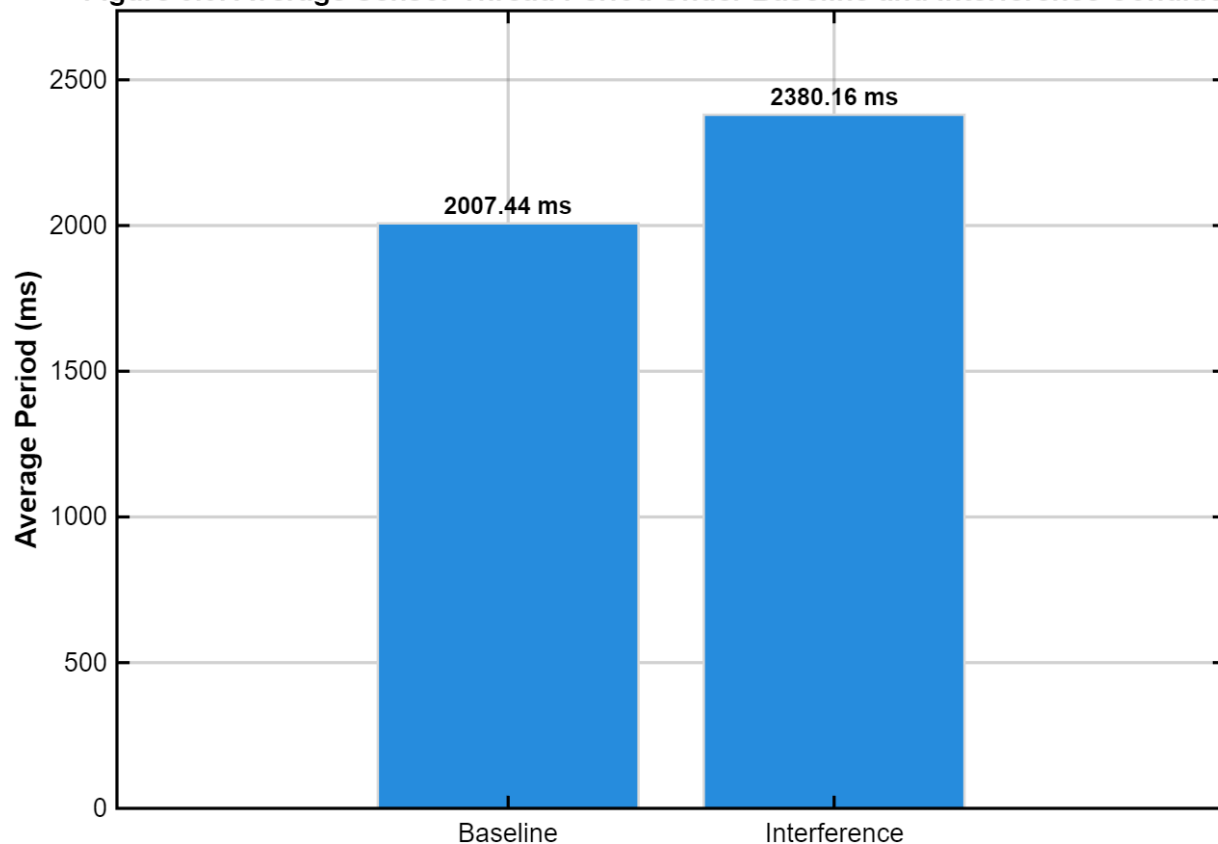
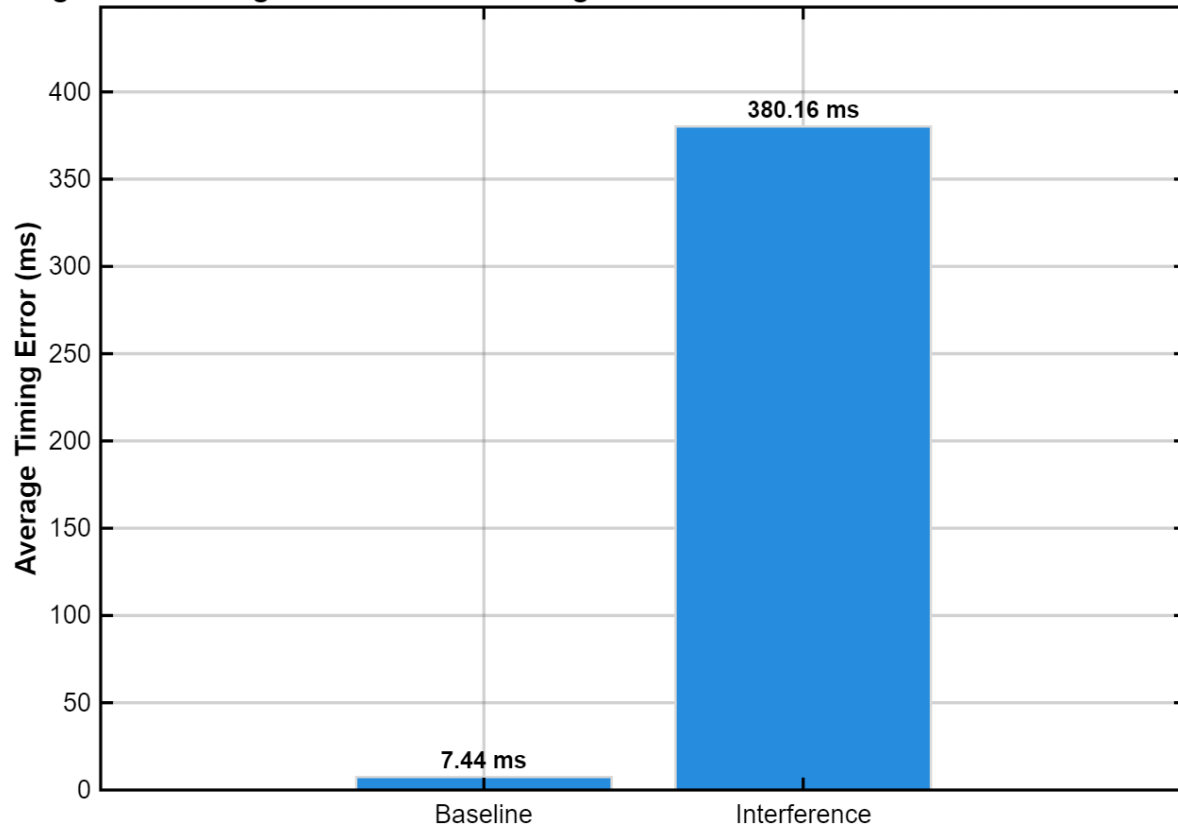


Figure 5.4. Average Sensor Thread Timing Error Under Baseline and Interference Conditions



5.7 Attempted Power Measurement

In addition to timing-related evaluation, the original project goals also included power-consumption measurement. Specifically, the goals called for current measurements in states such as idle, BLE send, and sensor-read operation.

5.7.1 Attempted measurement setup

In Phase 4, current measurement was tried using the **Nordic Power Profiler Kit II (PPK2)**. The board was powered and could still execute firmware while the profiler displayed a trace. This meant the experiment was not purely theoretical; the instrumentation process was tried on hardware.

5.7.2 Why the data was rejected

The measured values were implausibly small, in the sub-microamp range, even when the board was:

- running active firmware,
- printing serial output,
- and executing BLE and/or busy-thread behavior.

Those values were not credible representations of the true target current. A later attempt to alter the board-side current-measurement path also made the setup unreliable enough that the measurement effort was stopped.

5.7.3 Outcome of the Power Measurement Attempt

The power-consumption experiment is therefore documented as:

- attempted,
- incomplete,
- and excluded from formal quantitative conclusions.

This is the correct research decision. It is better to report an incomplete measurement honestly than to include numbers that are not trustworthy. Figure 5.5 shows a screenshot of the Nordic Power Profiler Kit II during the attempted current-measurement experiment, proving implausibly low current readings despite active firmware execution; this result was treated as untrustworthy and excluded from formal quantitative analysis.

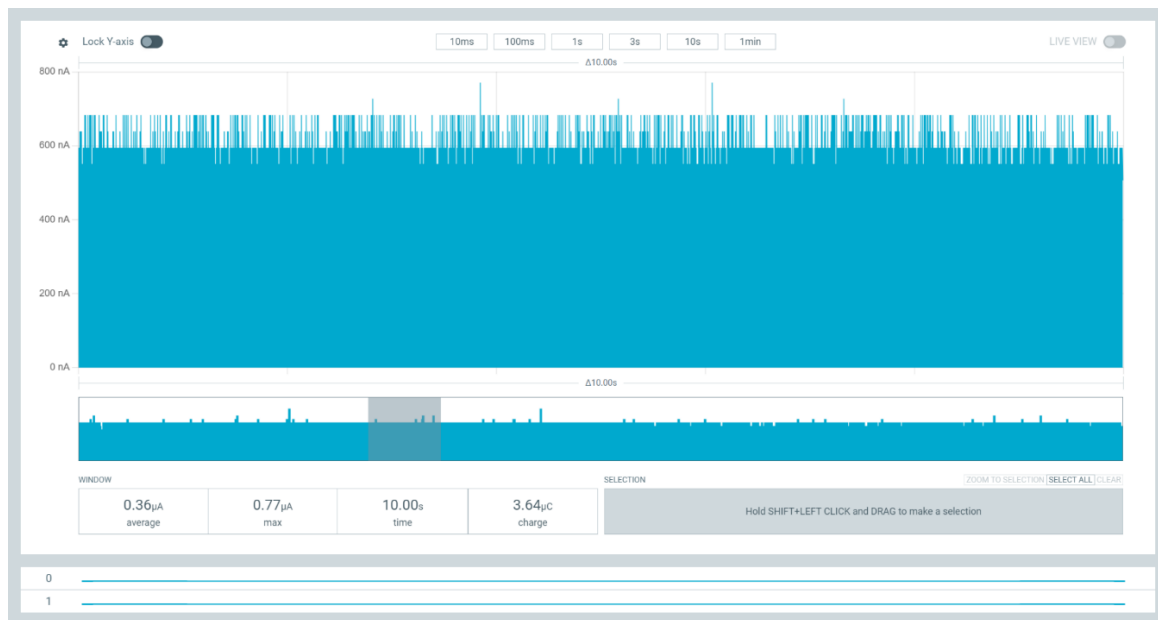


Figure 5.5: Screenshot of PPK2 measurement attempt showing implausible current readings.

5.8 Discussion of Results

The experimental results offer several important insights into the implemented Zephyr RTOS-based BLE sensing node.

5.8.1 The custom node is functionally valid and experimentally measurable

At the end of Phase 3 and Phase 4, the system had already shown:

- working BLE operation,
- a custom advertising node,
- a readable custom characteristic,
- and live application data changing on the board and visible at the client.

Phase 4 then showed that this same node could be instrumented to produce useful timing measurements. This is significant because it means the final prototype is not just a demo; it is a measurable research platform.

5.8.2 Application-level latency is strongly shaped by periodic update timing

The task-latency experiment showed a wide range of data ages at the time of BLE read, from 616 ms to 1927 ms. This is consistent with a system where:

- the board updates periodically,
- the client reads asynchronously,
- and the returned value may therefore be newer or older depending on where the read falls within the update cycle.

This highlights an important application-level truth for BLE sensing nodes: the useful latency experienced by a client is not always identical to radio-level transmission time. Data freshness and update timing matter.

5.8.3 Zephyr periodic scheduling was highly stable under baseline conditions

The baseline scheduler result was remarkably clean:

- 48 cycles,
- 2007–2008 ms actual period,
- only +7 to +8 ms average error,
- 1 ms jitter range.

This suggests that under light load, the Zephyr-based sensor task ran very predictably on the nRF52840 DK.

5.8.4 CPU contention produced delay more than disorder

The interference experiment is perhaps the strongest timing result in the project. The high-priority busy thread caused the sensor thread's average period to shift upward by about 372.72 ms, yet the jitter remained low.

That means the effect of interference was mainly:

- a systematic delay,
not:
- chaotic variability.

This distinction matters because it suggests that in the tested system, scheduler contention altered timing in a structured way. That is useful analytically, because predictable delay can be quantified and discussed more clearly than random instability.

5.8.5 The project is strong in timing results, weaker in power results

The timing experiments are well supported by:

- serial logs,
- summary tables,
- and graphs.

By contrast, the power experiment did not reach a publishable measurement state. This does not invalidate the project, but it does define one of its limitations.

5.9 Limitations

Several limitations should be acknowledged in the interpretation of the experimental results.

5.9.1 No validated power-consumption dataset

The most obvious limitation is that the power-consumption experiment was not completed with trustworthy data. Therefore, the project's performance characterization is strong on timing and scheduler behavior, but incomplete on electrical power behavior.

5.9.2 Simulated temperature source rather than physical sensor

The system used a simulated sensor value rather than a physical temperature sensor. This was allowed by the project goals and was sufficient for studying RTOS and BLE behavior, but it means the final prototype does not yet include physical sensing hardware integration.

5.9.3 Read-based BLE evaluation rather than streaming/notify behavior

The BLE interaction model used repeated client reads, not notifications or indications as the primary application data path in the custom node. This was suitable for the implemented experiments, but it means the measured latency reflects read-time data freshness rather than continuous push-based update behavior.

5.9.4 Limited sample counts

Although the timing datasets were sufficient for clear experimental conclusions, they were still relatively modest in size:

- 10 BLE reads for latency,
- 48 cycles for baseline scheduler timing,
- 45 cycles for interference timing.

A larger dataset could support stronger statistical treatment.

5.9.5 Single hardware platform

All experiments were performed on a single nRF52840 DK target setup. The conclusions are therefore grounded in that platform and toolchain configuration, not in cross-platform comparison.

5.10 Summary

This chapter presented the experimental results and discussion for the completed research project. The main results can be summarized as follows:

- Phase 2 demonstrated core scheduler concepts including starvation, synchronization, and equal-priority fairness.
- Phase 4A measured sensor-update-to-client-read latency and found values ranging from 616 ms to 1927 ms, with an average of 1323.5 ms.
- Phase 4B showed highly stable baseline periodic timing, with an average sensor-thread period of 2007.44 ms.
- Phase 4C showed that higher-priority CPU interference increased the average sensor-thread period to 2380.16 ms, a shift of 372.72 ms compared to baseline, while still preserving low jitter.
- Power measurement was tried but excluded from formal claims because the values obtained were not trustworthy.

Together, these results provide a strong timing-based characterization of the custom Zephyr RTOS BLE node and set up a solid basis for the project's final conclusions.

Chapter 6: Conclusion, Limitations, and Future Work

6.1 Conclusion

This research project investigated the design, implementation, validation, and timing-based evaluation of a Zephyr RTOS-based BLE sensing node on the Nordic nRF52840 Development Kit. The work began with complete development environment preparation on a Windows host platform, continued through RTOS experimentation and BLE validation, and culminated in a functioning custom BLE temperature-monitoring prototype with experimentally measured timing behavior.

The first major achievement of the project was successful Zephyr bring-up on the target platform. A complete build, flash, and runtime workflow was set up, including correct board targeting, SDK/toolchain integration, flashing support, and serial console verification on the nRF52840 DK. This foundational step was critical because it transformed the research from a planning exercise into a repeatable embedded development workflow.

The second major achievement was practical study of Zephyr RTOS behavior through custom multithreaded applications. The project did not stop at basic thread creation. Instead, it experimentally showed:

- sleep-based timing behavior,
- output corruption caused by unsynchronized shared UART access,
- correction of that problem through mutex protection,
- starvation under a continuously ready high-priority busy thread,
- the inadequacy of `k_yield()` alone in the tested priority scenario,
- restoration of lower-priority execution through blocking sleep,
- and balanced execution under equal-priority conditions.

The third major achievement was successful BLE validation and custom BLE node development. BLE functionality was first set up using the Zephyr peripheral sample and nRF Connect on a mobile device. After correcting a runtime stack overflow through configuration changes, the BLE stack ran successfully and advertised correctly. The project then advanced to a custom BLE application, *Phase3 Temp Node*, which generated simulated temperature values on the board, exposed them through a custom GATT service and characteristic, and returned changing values to the phone client.

The fourth major achievement was quantitative timing analysis. Phase 4 transformed the custom BLE node into a measurable research system. Application-level latency was measured as the time between the latest sensor update and the moment the BLE characteristic was read by the

client. That experiment produced latencies from 616 ms to 1927 ms with an average of 1323.5 ms. The scheduler timing experiments then showed that:

- under baseline conditions, the 2-second sensor thread remained highly stable with an average measured period of 2007.44 ms,
- under interference from a higher-priority busy thread, the average period shifted to 2380.16 ms,
- producing an average increase of 372.72 ms.

A particularly important conclusion from the timing experiments is that interference did not primarily manifest as large random jitter. Instead, it imposed a substantial and repeatable delay. This is a meaningful result because it shows that the Zephyr scheduler on the target platform remained behaviorally structured under contention, even while lower-priority periodic work experienced a large timing shift.

Overall, this research project shows that it is possible to build and experimentally evaluate a Zephyr RTOS-based BLE sensing node on embedded hardware using a phased workflow:

- environment preparation,
- board validation,
- RTOS experimentation,
- BLE validation,
- custom node implementation,
- and timing-performance analysis.

The project did not merely demonstrate that Zephyr and BLE can run on the nRF52840 DK. It also showed how RTOS-level scheduling behavior affects the timing behavior of an application-level BLE sensing system, which is the project's main research contribution.

6.2 Limitations

Despite the project's successful implementation and timing evaluation, several limitations remain.

6.2.1 Incomplete power-consumption evaluation

The most important limitation is that the power-consumption part of the performance study was not completed with trustworthy data. The Nordic Power Profiler Kit II was used to measure current under different firmware states, but the resulting values were not credible representations

of actual target current. As a result, that part of Goal 8 was documented as attempted but incomplete, and the current traces were excluded from formal quantitative conclusions.

6.2.2 Simulated sensing rather than physical sensor integration

The final custom node used simulated temperature data rather than a physical onboard or external temperature sensor. This was allowed by the project goals and was sufficient for studying RTOS timing and BLE data exposure, but it means the final prototype does not yet include validation of real sensor acquisition hardware in the data path.

6.2.3 BLE data model limited to read-based interaction

The custom BLE system was verified primarily through client-initiated characteristic reads rather than through a notification- or indication-driven update model. This made the Phase 4 latency experiment straightforward and interpretable, but it also means the system was not evaluated as a continuously streaming or push-driven BLE sensor node.

6.2.4 Limited scope of timing experiments

The timing experiments were strong enough to support meaningful conclusions, but they were still modest in scale. The final quantitative datasets included:

- 10 BLE latency reads,
- 48 baseline scheduler cycles,
- 45 interference scheduler cycles.

These sample sizes were sufficient to show clear trends, but a larger dataset could support stronger statistical analysis.

6.2.5 No completed smartphone-to-cloud gateway implementation

The system architecture was interpreted practically as BLE to smartphone to Wi-Fi/cloud, but the project stopped at the BLE node and phone-client stage. The smartphone/gateway/cloud part stays conceptual rather than fully implemented. This means the project should be understood as a BLE sensing-node study, not as a finished end-to-end IoT cloud platform.

6.2.6 Platform-specific conclusions

All implementation and experiments were conducted on the Nordic nRF52840 Development Kit under a specific Zephyr toolchain and Windows-host workflow. The conclusions are therefore grounded in this platform combination and should not automatically be generalized to all Zephyr-supported boards or all embedded RTOS environments without further study.

6.3 Future Work

Several directions can extend and strengthen this research.

6.3.1 Reliable power-consumption measurement

The clearest next step is to complete the power-consumption experiment using a validated and supervised measurement setup. Since the timing part of Goal 8 is already strong, a trustworthy power dataset would round out the project by adding the missing electrical-performance dimension. This could include current measurements for:

- idle operation,
- BLE advertising,
- BLE connection and read activity,
- sensor-update activity,
- busy-thread interference conditions.

6.3.2 Physical sensor integration

A future version of the project could replace the simulated temperature source with a real onboard or external sensor. This would make the system more representative of a practical sensing application and would allow study of:

- actual sensor acquisition timing,
- sensor-driver overhead,
- and end-to-end timing from physical measurement to BLE exposure.

6.3.3 BLE notifications or indications

The current system uses read-based BLE access. Future work could add a notify- or indicate-based characteristic update model so that data is pushed from the node to the client rather than requested manually. That would make it possible to study:

- update-delivery latency under a push model,
- BLE traffic patterns under continuous updates,
- and potentially tighter coupling between periodic sensor updates and seen client data.

6.3.4 Gateway and cloud integration

The goals already described the intended broader architecture as BLE to smartphone to Wi-Fi to cloud. A future extension could therefore implement:

- a smartphone gateway application,

- data forwarding to a remote server,
- timestamped cloud logging of BLE sensor values,
- and comparative measurements of local BLE latency versus full end-to-end IoT path latency.

6.3.5 Broader scheduler experimentation

The scheduler results in this project were strong, but future work could expand them by testing:

- added thread-priority combinations,
- different sleep intervals,
- more than one competing interference thread,
- semaphore- or queue-based task coordination,
- and timing under heavier BLE activity.

This would deepen the RTOS research dimension of the work.

6.3.6 Cross-platform comparison

Future research could repeat the same system and experiments on another Zephyr-supported board or compare Zephyr with another embedded RTOS. That would help separate platform-specific findings from more general scheduler or BLE-behavior conclusions.

6.4 Final Remarks

This research project successfully progressed from development-environment bring-up to implementation of a functioning BLE sensing node and then to timing-based performance characterization on real embedded hardware. The work shows that Zephyr RTOS is not only a usable embedded development platform for the nRF52840 DK, but also a practical research platform for studying real scheduler behavior, concurrency effects, and application-level wireless timing.

Perhaps the most important result is that the project moved beyond simple functional validation. It did not stop at “the board runs code” or “the phone can connect over BLE.” Instead, it used those achievements as a basis for experimental study. The outcome is therefore both an implemented system and a documented set of timing results that support meaningful conclusions about the interaction between RTOS scheduling and BLE-based sensing behavior.

Even with the incomplete power-measurement part, the project stands as a successful embedded research study. It provides a documented workflow, a working prototype, a timing

analysis dataset, and a clear path for future extensions involving stronger instrumentation, physical sensing, and more complete IoT integration.

References

- [1] Zephyr Project, “Getting Started Guide,” *Zephyr Documentation*, [Online]. Available: https://docs.zephyrproject.org/latest/develop/getting_started/index.html. [Accessed: May 2026].
- [2] Zephyr Project, “West (Zephyr’s meta-tool),” *Zephyr Documentation*, [Online]. Available: <https://docs.zephyrproject.org/latest/develop/west/index.html>. [Accessed: May 2026].
- [3] Zephyr Project, “Bluetooth,” *Zephyr Documentation*, [Online]. Available: <https://docs.zephyrproject.org/latest/services/connectivity/bluetooth/index.html>. [Accessed: May 2026].
- [4] Zephyr Project, “Bluetooth Samples,” *Zephyr Documentation*, [Online]. Available: <https://docs.zephyrproject.org/latest/samples/bluetooth/bluetooth.html>. [Accessed: May 2026].
- [5] Zephyr Project, *Zephyr Project Documentation*, [Online]. Available: <https://docs.zephyrproject.org/latest/index.html>. [Accessed: May 2026].
- [6] Nordic Semiconductor, “nRF52840 DK Hardware,” *Technical Documentation*, [Online]. Available: https://docs.nordicsemi.com/bundle/ug_nrf52840_dk/page/UG/dk/intro.html. [Accessed: May 2026].
- [7] Nordic Semiconductor, “Getting Started,” *nRF52840 DK Technical Documentation*, [Online]. Available: https://docs.nordicsemi.com/bundle/ug_nrf52840_dk/page/UG/dk/getting_started.html. [Accessed: May 2026].

[8] Nordic Semiconductor, “Set up the DK,” *nRF52840 DK Technical Documentation*, [Online].

Available:

https://docs.nordicsemi.com/bundle/ug_nrf52840_dk/page/UG/dk/prepare_board.html.

[Accessed: May 2026].

[9] Nordic Semiconductor, “nRF52840 DK Development Kit,” *Nordic Semiconductor*, [Online].

Available: <https://www.nordicsemi.com/Products/Development-hardware/nRF52840-DK>.

[Accessed: May 2026].

[10] Bluetooth SIG, Inc., “Bluetooth® Technology Website,” [Online]. Available:

<https://www.bluetooth.com/>. [Accessed: May 2026].

[11] A. Eliaz, *Zephyr RTOS Embedded C Programming*. New York, NY, USA: Apress, 2024.

doi: 10.1007/979-8-8688-0107-5.

[12] R. P. Gangarapu and P. Prithvi, “Cloud-Integrated BLE Temperature Control and Monitoring System for Portable Transport of Temperature Sensitive Goods,” Dept. of ECE, National Institute of Technology Warangal, Warangal, Telangana, India, 2025.

[13] G. M. Torres, J. V. L. De Souza, J. G. F. Aguiar, and J. O. De Sousa, “Performance and Power Consumption of Real-Time Task Scheduling for Embedded Systems Devices,” 2025.

[14] C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[15] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011.

Appendices

Appendix A: Phase 2 Scheduler Experiment Code

File: *8_Experiment_1C_equal_priorities_main.c*

This file has the source code used for the **Phase 2 equal-priority scheduler experiment**. The program creates two worker threads, `THREAD_A` and `THREAD_B`, each assigned to the same Zephyr priority level (5) and the same periodic sleep interval of 1000 ms. A mutex-protected `safe_print()` function is used to serialize UART output and avoid interleaved console messages during concurrent execution. The main thread stays active separately and prints a status message every 3000 ms.

The purpose of this experiment was to see Zephyr scheduling behavior when two runnable threads have **equal priority and identical blocking behavior**. Because both threads sleep for the same interval and use the same priority, the output shows balanced and repeatable execution rather than starvation. This file was selected for the appendix because it stands for one of the clearest scheduler-validation experiments from Phase 2 and directly supports the discussion of **fairness under equal-priority conditions**.

Content:

```
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>

#define STACK_SIZE 1024

void thread_a(void *, void *, void *);
void thread_b(void *, void *, void *);

K_THREAD_STACK_DEFINE(stack_a, STACK_SIZE);
K_THREAD_STACK_DEFINE(stack_b, STACK_SIZE);

struct k_thread thread_a_data;
struct k_thread thread_b_data;

K_MUTEX_DEFINE(print_mutex);

static void safe_print(const char *thread_name, const char *message)
{
    int64_t ms = k_uptime_get();

    k_mutex_lock(&print_mutex, K_FOREVER);
```

```

    printk("[%lld ms] %s: %s\n", ms, thread_name, message);
    k_mutex_unlock(&print_mutex);
}

void thread_a(void *arg1, void *arg2, void *arg3)
{
    ARG_UNUSED(arg1);
    ARG_UNUSED(arg2);
    ARG_UNUSED(arg3);

    while (1) {
        safe_print("THREAD_A", "running, then sleeping 1000 ms");
        k_sleep(K_SECONDS(1));
    }
}

void thread_b(void *arg1, void *arg2, void *arg3)
{
    ARG_UNUSED(arg1);
    ARG_UNUSED(arg2);
    ARG_UNUSED(arg3);

    while (1) {
        safe_print("THREAD_B", "running, then sleeping 1000 ms");
        k_sleep(K_SECONDS(1));
    }
}

int main(void)
{
    safe_print("MAIN", "Phase 2 - Experiment 1C start");

    k_thread_create(&thread_a_data, stack_a, STACK_SIZE,
                    thread_a,
                    NULL, NULL, NULL,
                    5, 0, K_NO_WAIT);

    k_thread_create(&thread_b_data, stack_b, STACK_SIZE,
                    thread_b,
                    NULL, NULL, NULL,

```

```
    5, 0, K_NO_WAIT);

while (1) {
    safe_print("MAIN", "main thread running, then sleeping 3000 ms");
    k_sleep(K_SECONDS(3));
}

return 0;
}
```

Appendix B: Custom BLE Temperature Node Source Code

File: *5_phase3_goal5_build_success_main.c*

This file has the source code for the **custom BLE temperature node** implemented in Phase 3. The program initializes the Zephyr Bluetooth stack, loads BLE settings, starts connectable advertising, and defines a custom GATT service with a readable temperature characteristic. A separate sensor thread periodically updates a simulated temperature value every 2 seconds using pseudo-random values in the range of approximately **20.0 °C to 34.9 °C**. Access to the shared temperature variable is protected using a Zephyr mutex to ensure thread-safe reads and writes.

This file is important because it stands for the first complete implementation of the project's **custom research system**: a working Zephyr RTOS-based BLE node that advertises successfully, exposes application data through a custom service, and can be read by a mobile BLE client. It also reflects several practical implementation decisions discussed in the main body of the report, including reuse of a known-good advertising configuration (BT_LE_ADV_CONN_FAST_1) and use of simulated sensing in place of physical sensor hardware.

Content:

```
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>
#include <zephyr/bluetooth/bluetooth.h>
#include <zephyr/bluetooth/gatt.h>
#include <zephyr/settings/settings.h>
#include <zephyr/random/random.h>
#include <string.h>

#define STACK_SIZE 1024
#define SENSOR_PRIORITY 5

static int16_t temperature_c = 250; /* 25.0 C in tenths of a degree */

K_THREAD_STACK_DEFINE(sensor_stack, STACK_SIZE);
struct k_thread sensor_thread_data;

K_MUTEX_DEFINE(temp_mutex);

static ssize_t read_temp(struct bt_conn *conn,
                        const struct bt_gatt_attr *attr,
                        void *buf,
                        uint16_t len,
                        uint16_t offset)
```

```

{
    int16_t temp_copy;

    ARG_UNUSED(conn);

    k_mutex_lock(&temp_mutex, K_FOREVER);
    temp_copy = temperature_c;
    k_mutex_unlock(&temp_mutex);

    return bt_gatt_attr_read(conn, attr, buf, len, offset,
                             &temp_copy, sizeof(temp_copy));
}

/* Custom service UUID: 12345678-1234-5678-1234-56789abcdef0 */
/* Custom temp characteristic UUID: 12345678-1234-5678-1234-56789abcdef1 */

BT_GATT_SERVICE_DEFINE(temp_svc,
    BT_GATT_PRIMARY_SERVICE(BT_UUID_DECLARE_128(
        BT_UUID_128_ENCODE(0x12345678, 0x1234, 0x5678, 0x1234, 0x56789abcdef0))),
    BT_GATT_CHARACTERISTIC(BT_UUID_DECLARE_128(
        BT_UUID_128_ENCODE(0x12345678, 0x1234, 0x5678, 0x1234, 0x56789abcdef1)),
        BT_GATT_CHRC_READ,
        BT_GATT_PERM_READ,
        read_temp, NULL, NULL)
);

static const struct bt_data ad[] = {
    BT_DATA_BYTES(BT_DATA_FLAGS, (BT_LE_AD_GENERAL |
BT_LE_AD_NO_BREDR)),
    BT_DATA(BT_DATA_NAME_COMPLETE, CONFIG_BT_DEVICE_NAME,
        sizeof(CONFIG_BT_DEVICE_NAME) - 1),
};

void sensor_thread(void *arg1, void *arg2, void *arg3)
{
    ARG_UNUSED(arg1);
    ARG_UNUSED(arg2);
    ARG_UNUSED(arg3);

    while (1) {

```

```

int16_t new_temp;

/* Simulate temperature from 20.0 C to 34.9 C */
new_temp = 200 + (sys_rand32_get() % 150);

k_mutex_lock(&temp_mutex, K_FOREVER);
temperature_c = new_temp;
k_mutex_unlock(&temp_mutex);

printk("Sensor thread updated temperature: %d.%d C\n",
       new_temp / 10,
       (new_temp < 0) ? -(new_temp % 10) : (new_temp % 10));

k_sleep(K_SECONDS(2));
}
}

int main(void)
{
    int err;

    printk("Phase 3 Goal 5: Custom BLE temperature node starting\n");

    err = bt_enable(NULL);
    if (err) {
        printk("Bluetooth init failed (err %d)\n", err);
        return 0;
    }

    printk("Bluetooth initialized\n");

    err = settings_load();
    if (err) {
        printk("settings_load failed (err %d)\n", err);
    } else {
        printk("Settings loaded\n");
    }

    err = bt_le_adv_start(BT_LE_ADV_CONN_FAST_1, ad, ARRAY_SIZE(ad), NULL, 0);

```

```
if (err) {
    printk("Advertising failed to start (err %d)\n", err);
    return 0;
}

printk("Advertising successfully started\n");

k_thread_create(&sensor_thread_data, sensor_stack, STACK_SIZE,
               sensor_thread,
               NULL, NULL, NULL,
               SENSOR_PRIORITY, 0, K_NO_WAIT);

while (1) {
    k_sleep(K_SECONDS(5));
}

return 0;
}
```


Appendix C: Raw Timing Logs

Appendix C.1:

File: *phase4A_latency_log.txt*

This log file has the raw serial output for **Phase 4A**, the application-level task latency experiment. The log records repeated sensor updates together with ten BLE characteristic reads performed from the mobile client. For each BLE read, the program prints the current temperature value and the calculated latency between the most recent sensor update and the client read event. These measurements form the basis of the **sensor-update-to-client-read latency analysis** discussed in Chapter 5. The recorded latencies range from **616 ms to 1927 ms**, showing that the freshness of returned BLE data depends on when the client reads the characteristic compared to the 2-second sensor update cycle.

Content:

```
Sensor update -> temp=26.8 C at 517229 ms
Sensor update -> temp=24.7 C at 519232 ms
Sensor update -> temp=33.6 C at 521236 ms
Sensor update -> temp=29.5 C at 523240 ms
Sensor update -> temp=30.3 C at 525244 ms
Sensor update -> temp=23.2 C at 527247 ms
Sensor update -> temp=21.8 C at 529251 ms
Sensor update -> temp=23.1 C at 531255 ms
Sensor update -> temp=23.7 C at 533259 ms
Sensor update -> temp=24.4 C at 535262 ms
Sensor update -> temp=30.1 C at 537266 ms
Sensor update -> temp=20.7 C at 539270 ms
Sensor update -> temp=28.8 C at 541273 ms
Sensor update -> temp=23.5 C at 543277 ms
Sensor update -> temp=31.3 C at 545281 ms
Sensor update -> temp=31.4 C at 547285 ms
Sensor update -> temp=23.5 C at 549288 ms
Sensor update -> temp=32.9 C at 551292 ms
Sensor update -> temp=31.5 C at 553296 ms
Sensor update -> temp=29.6 C at 555300 ms
Sensor update -> temp=30.6 C at 557303 ms
Sensor update -> temp=24.2 C at 559307 ms
Sensor update -> temp=33.4 C at 561311 ms
Sensor update -> temp=21.7 C at 563315 ms
Sensor update -> temp=22.4 C at 565318 ms
```

Sensor update -> temp=20.4 C at 567322 ms
Sensor update -> temp=22.1 C at 569326 ms
Sensor update -> temp=33.9 C at 571330 ms
Sensor update -> temp=22.5 C at 573333 ms
Sensor update -> temp=30.7 C at 575337 ms
Sensor update -> temp=23.5 C at 577341 ms
Sensor update -> temp=29.0 C at 579345 ms
Sensor update -> temp=25.1 C at 581348 ms
Sensor update -> temp=22.8 C at 583352 ms
Sensor update -> temp=31.0 C at 585356 ms
Sensor update -> temp=26.8 C at 587360 ms
Sensor update -> temp=29.0 C at 589363 ms
Sensor update -> temp=31.7 C at 591367 ms
Sensor update -> temp=25.5 C at 593371 ms
Sensor update -> temp=30.3 C at 595374 ms
Sensor update -> temp=22.7 C at 597378 ms
Sensor update -> temp=25.1 C at 599382 ms
Sensor update -> temp=29.4 C at 601386 ms
Sensor update -> temp=27.1 C at 603389 ms
Sensor update -> temp=20.8 C at 605393 ms
Sensor update -> temp=28.9 C at 607397 ms
BLE read #1 -> temp=28.9 C, latency since sensor update = 1783 ms
Sensor update -> temp=29.7 C at 609401 ms
Sensor update -> temp=23.9 C at 611404 ms
Sensor update -> temp=21.3 C at 613408 ms
Sensor update -> temp=34.8 C at 615412 ms
Sensor update -> temp=24.4 C at 617416 ms
Sensor update -> temp=30.3 C at 619419 ms
Sensor update -> temp=33.8 C at 621423 ms
Sensor update -> temp=29.9 C at 623427 ms
Sensor update -> temp=23.8 C at 625431 ms
Sensor update -> temp=23.5 C at 627434 ms
Sensor update -> temp=24.3 C at 629438 ms
Sensor update -> temp=24.4 C at 631442 ms
Sensor update -> temp=34.2 C at 633445 ms
Sensor update -> temp=27.2 C at 635449 ms
BLE read #2 -> temp=27.2 C, latency since sensor update = 1151 ms
Sensor update -> temp=29.0 C at 637453 ms
Sensor update -> temp=23.4 C at 639457 ms
Sensor update -> temp=33.9 C at 641460 ms

Sensor update -> temp=26.9 C at 643464 ms
Sensor update -> temp=31.5 C at 645468 ms
BLE read #3 -> temp=31.5 C, latency since sensor update = 1122 ms
Sensor update -> temp=23.6 C at 647472 ms
Sensor update -> temp=33.0 C at 649475 ms
Sensor update -> temp=26.7 C at 651479 ms
Sensor update -> temp=22.8 C at 653483 ms
Sensor update -> temp=23.6 C at 655487 ms
Sensor update -> temp=23.8 C at 657490 ms
Sensor update -> temp=21.2 C at 659494 ms
Sensor update -> temp=26.8 C at 661498 ms
Sensor update -> temp=25.5 C at 663502 ms
BLE read #4 -> temp=25.5 C, latency since sensor update = 1927 ms
Sensor update -> temp=22.1 C at 665505 ms
Sensor update -> temp=20.5 C at 667509 ms
Sensor update -> temp=32.2 C at 669513 ms
Sensor update -> temp=27.1 C at 671516 ms
Sensor update -> temp=27.1 C at 673520 ms
Sensor update -> temp=25.0 C at 675524 ms
Sensor update -> temp=21.0 C at 677528 ms
Sensor update -> temp=31.3 C at 679531 ms
Sensor update -> temp=27.6 C at 681535 ms
Sensor update -> temp=21.1 C at 683539 ms
Sensor update -> temp=26.8 C at 685543 ms
BLE read #5 -> temp=26.8 C, latency since sensor update = 946 ms
Sensor update -> temp=21.2 C at 687546 ms
Sensor update -> temp=33.3 C at 689550 ms
Sensor update -> temp=31.1 C at 691554 ms
Sensor update -> temp=33.0 C at 693558 ms
Sensor update -> temp=30.2 C at 695561 ms
Sensor update -> temp=25.7 C at 697565 ms
Sensor update -> temp=23.1 C at 699569 ms
Sensor update -> temp=24.2 C at 701572 ms
Sensor update -> temp=32.3 C at 703576 ms
BLE read #6 -> temp=32.3 C, latency since sensor update = 1453 ms
Sensor update -> temp=24.7 C at 705580 ms
Sensor update -> temp=22.3 C at 707584 ms
Sensor update -> temp=33.4 C at 709587 ms
Sensor update -> temp=21.3 C at 711591 ms
Sensor update -> temp=28.2 C at 713595 ms

Sensor update -> temp=22.1 C at 715599 ms
Sensor update -> temp=23.9 C at 717602 ms
Sensor update -> temp=30.7 C at 719606 ms
BLE read #7 -> temp=30.7 C, latency since sensor update = 1142 ms
Sensor update -> temp=25.8 C at 721610 ms
Sensor update -> temp=20.3 C at 723614 ms
Sensor update -> temp=20.1 C at 725617 ms
Sensor update -> temp=34.5 C at 727621 ms
Sensor update -> temp=31.1 C at 729625 ms
Sensor update -> temp=22.6 C at 731629 ms
Sensor update -> temp=21.0 C at 733632 ms
BLE read #8 -> temp=21.0 C, latency since sensor update = 616 ms
Sensor update -> temp=26.0 C at 735636 ms
Sensor update -> temp=33.1 C at 737640 ms
Sensor update -> temp=20.5 C at 739643 ms
Sensor update -> temp=28.4 C at 741647 ms
Sensor update -> temp=25.2 C at 743651 ms
Sensor update -> temp=28.3 C at 745655 ms
BLE read #9 -> temp=28.3 C, latency since sensor update = 1193 ms
Sensor update -> temp=32.2 C at 747658 ms
Sensor update -> temp=21.9 C at 749662 ms
Sensor update -> temp=24.0 C at 751666 ms
Sensor update -> temp=32.6 C at 753670 ms
Sensor update -> temp=28.4 C at 755673 ms
Sensor update -> temp=30.9 C at 757677 ms
Sensor update -> temp=27.6 C at 759681 ms
Sensor update -> temp=26.3 C at 761685 ms
BLE read #10 -> temp=26.3 C, latency since sensor update = 1902 ms
Sensor update -> temp=21.6 C at 763688 ms
Sensor update -> temp=28.1 C at 765692 ms
Sensor update -> temp=34.8 C at 767696 ms
Sensor update -> temp=28.8 C at 769699 ms
Sensor update -> temp=29.8 C at 771703 ms
Sensor update -> temp=24.4 C at 773707 ms
Sensor update -> temp=20.0 C at 775711 ms
Sensor update -> temp=27.8 C at 777714 ms
Sensor update -> temp=26.3 C at 779718 ms
Sensor update -> temp=21.4 C at 781722 ms
Sensor update -> temp=28.9 C at 783726 ms
Sensor update -> temp=23.3 C at 785729 ms

Sensor update -> temp=20.9 C at 787733 ms
 Sensor update -> temp=30.5 C at 789737 ms
 Sensor update -> temp=22.8 C at 791741 ms
 Sensor update -> temp=28.1 C at 793744 ms
 Sensor update -> temp=27.6 C at 795748 ms
 Sensor update -> temp=30.1 C at 797752 ms
 Sensor update -> temp=23.3 C at 799756 ms
 Sensor update -> temp=26.9 C at 801759 ms
 Sensor update -> temp=27.4 C at 803763 ms
 Sensor update -> temp=24.5 C at 805767 ms
 Sensor update -> temp=21.5 C at 807770 ms
 Sensor update -> temp=29.8 C at 809774 ms
 Sensor update -> temp=31.1 C at 811778 ms
 Sensor update -> temp=33.4 C at 813782 ms
 Sensor update -> temp=31.8 C at 815785 ms
 Sensor update -> temp=33.9 C at 817789 ms

Appendix C.2:

File: *phase4B_scheduler_log.txt*

This log file has the raw serial output for **Phase 4B**, the baseline scheduler timing experiment. Each line records a sensor update cycle, the measured timestamp, the actual execution period since the earlier cycle, and the timing error compared to the intended 2000 ms interval. The log shows that under baseline conditions the periodic sensor thread remained highly stable, with measured periods consistently alternating between **2007 ms and 2008 ms** and timing error staying between **+7 ms and +8 ms**. This file provides raw evidence supporting the report's conclusion that the Zephyr periodic task showed **low jitter and strong timing stability under light load**.

Content:

Sensor cycle #31 -> temp=20.7 C at 60488 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #32 -> temp=32.1 C at 62496 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #33 -> temp=28.7 C at 64503 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #34 -> temp=30.2 C at 66510 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #35 -> temp=29.1 C at 68518 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #36 -> temp=31.4 C at 70525 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #37 -> temp=25.2 C at 72532 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #38 -> temp=32.0 C at 74540 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #39 -> temp=27.3 C at 76547 ms, actual period = 2007 ms, error = +7 ms

Sensor cycle #40 -> temp=28.8 C at 78554 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #41 -> temp=34.7 C at 80562 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #42 -> temp=23.5 C at 82569 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #43 -> temp=26.2 C at 84576 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #44 -> temp=24.3 C at 86584 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #45 -> temp=23.5 C at 88591 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #46 -> temp=27.4 C at 90599 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #47 -> temp=24.2 C at 92606 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #48 -> temp=33.6 C at 94613 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #49 -> temp=34.9 C at 96621 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #50 -> temp=26.6 C at 98628 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #51 -> temp=29.2 C at 100635 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #52 -> temp=32.3 C at 102643 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #53 -> temp=21.2 C at 104650 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #54 -> temp=29.5 C at 106658 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #55 -> temp=32.9 C at 108665 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #56 -> temp=32.2 C at 110673 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #57 -> temp=28.8 C at 112680 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #58 -> temp=27.8 C at 114688 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #59 -> temp=34.7 C at 116695 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #60 -> temp=23.6 C at 118703 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #61 -> temp=34.8 C at 120710 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #62 -> temp=27.0 C at 122718 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #63 -> temp=23.8 C at 124725 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #64 -> temp=33.1 C at 126733 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #65 -> temp=34.3 C at 128740 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #66 -> temp=22.2 C at 130748 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #67 -> temp=20.1 C at 132755 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #68 -> temp=24.5 C at 134763 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #69 -> temp=24.0 C at 136770 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #70 -> temp=23.2 C at 138778 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #71 -> temp=34.3 C at 140785 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #72 -> temp=21.0 C at 142793 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #73 -> temp=29.8 C at 144800 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #74 -> temp=33.9 C at 146807 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #75 -> temp=32.7 C at 148815 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #76 -> temp=34.6 C at 150822 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #77 -> temp=28.9 C at 152830 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #78 -> temp=23.5 C at 154837 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #79 -> temp=34.1 C at 156845 ms, actual period = 2008 ms, error = +8 ms

Sensor cycle #80 -> temp=30.2 C at 158852 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #81 -> temp=26.1 C at 160860 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #82 -> temp=30.0 C at 162867 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #83 -> temp=26.6 C at 164875 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #84 -> temp=31.9 C at 166882 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #85 -> temp=30.2 C at 168889 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #86 -> temp=20.7 C at 170897 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #87 -> temp=31.1 C at 172904 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #88 -> temp=26.5 C at 174912 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #89 -> temp=22.7 C at 176919 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #90 -> temp=27.1 C at 178927 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #91 -> temp=25.4 C at 180934 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #92 -> temp=22.3 C at 182942 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #93 -> temp=26.0 C at 184949 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #94 -> temp=29.7 C at 186957 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #95 -> temp=25.3 C at 188964 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #96 -> temp=26.9 C at 190971 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #97 -> temp=33.6 C at 192979 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #98 -> temp=20.9 C at 194986 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #99 -> temp=33.9 C at 196994 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #100 -> temp=26.6 C at 199001 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #101 -> temp=30.1 C at 201009 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #102 -> temp=33.3 C at 203017 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #103 -> temp=31.5 C at 205024 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #104 -> temp=21.5 C at 207032 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #105 -> temp=33.1 C at 209039 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #106 -> temp=27.7 C at 211047 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #107 -> temp=20.3 C at 213054 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #108 -> temp=27.6 C at 215062 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #109 -> temp=26.1 C at 217069 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #110 -> temp=21.9 C at 219077 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #111 -> temp=23.6 C at 221084 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #112 -> temp=32.5 C at 223092 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #113 -> temp=22.9 C at 225099 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #114 -> temp=27.2 C at 227107 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #115 -> temp=27.9 C at 229114 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #116 -> temp=32.5 C at 231122 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #117 -> temp=21.6 C at 233130 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #118 -> temp=32.7 C at 235137 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #119 -> temp=20.3 C at 237145 ms, actual period = 2008 ms, error = +8 ms

Sensor cycle #120 -> temp=33.3 C at 239152 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #121 -> temp=34.7 C at 241160 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #122 -> temp=20.9 C at 243168 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #123 -> temp=23.2 C at 245175 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #124 -> temp=22.0 C at 247183 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #125 -> temp=28.9 C at 249190 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #126 -> temp=32.4 C at 251198 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #127 -> temp=28.8 C at 253205 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #128 -> temp=22.0 C at 255213 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #129 -> temp=25.8 C at 257221 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #130 -> temp=29.9 C at 259228 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #131 -> temp=28.3 C at 261236 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #132 -> temp=21.2 C at 263243 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #133 -> temp=26.2 C at 265251 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #134 -> temp=29.7 C at 267258 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #135 -> temp=25.4 C at 269266 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #136 -> temp=29.5 C at 271273 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #137 -> temp=30.0 C at 273281 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #138 -> temp=31.5 C at 275288 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #139 -> temp=29.4 C at 277296 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #140 -> temp=31.8 C at 279303 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #141 -> temp=24.9 C at 281311 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #142 -> temp=27.2 C at 283319 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #143 -> temp=27.7 C at 285326 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #144 -> temp=31.0 C at 287334 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #145 -> temp=28.6 C at 289341 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #146 -> temp=31.3 C at 291349 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #147 -> temp=26.8 C at 293356 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #148 -> temp=29.6 C at 295364 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #149 -> temp=32.0 C at 297371 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #150 -> temp=34.8 C at 299379 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #151 -> temp=21.3 C at 301386 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #152 -> temp=25.0 C at 303394 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #153 -> temp=23.3 C at 305401 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #154 -> temp=34.2 C at 307409 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #155 -> temp=25.3 C at 309416 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #156 -> temp=29.0 C at 311424 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #157 -> temp=34.9 C at 313432 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #158 -> temp=22.8 C at 315439 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #159 -> temp=30.2 C at 317447 ms, actual period = 2008 ms, error = +8 ms

Sensor cycle #160 -> temp=28.8 C at 319454 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #161 -> temp=20.5 C at 321462 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #162 -> temp=26.0 C at 323469 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #163 -> temp=34.7 C at 325477 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #164 -> temp=20.5 C at 327484 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #165 -> temp=29.5 C at 329492 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #166 -> temp=26.2 C at 331499 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #167 -> temp=21.1 C at 333507 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #168 -> temp=23.6 C at 335514 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #169 -> temp=29.2 C at 337522 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #170 -> temp=25.4 C at 339529 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #171 -> temp=26.4 C at 341537 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #172 -> temp=24.0 C at 343544 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #173 -> temp=33.3 C at 345552 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #174 -> temp=27.5 C at 347559 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #175 -> temp=34.4 C at 349567 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #176 -> temp=24.2 C at 351575 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #177 -> temp=24.2 C at 353582 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #178 -> temp=25.9 C at 355590 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #179 -> temp=22.9 C at 357597 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #180 -> temp=32.8 C at 359605 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #181 -> temp=29.3 C at 361613 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #182 -> temp=31.9 C at 363620 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #183 -> temp=33.4 C at 365628 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #184 -> temp=21.4 C at 367635 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #185 -> temp=27.8 C at 369643 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #186 -> temp=23.0 C at 371650 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #187 -> temp=23.4 C at 373658 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #188 -> temp=21.8 C at 375665 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #189 -> temp=20.0 C at 377673 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #190 -> temp=31.5 C at 379680 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #191 -> temp=31.5 C at 381688 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #192 -> temp=32.3 C at 383696 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #193 -> temp=34.5 C at 385703 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #194 -> temp=21.4 C at 387711 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #195 -> temp=26.8 C at 389718 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #196 -> temp=24.5 C at 391726 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #197 -> temp=33.7 C at 393733 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #198 -> temp=25.7 C at 395741 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #199 -> temp=25.4 C at 397748 ms, actual period = 2007 ms, error = +7 ms

Sensor cycle #200 -> temp=34.4 C at 399756 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #201 -> temp=25.0 C at 401763 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #202 -> temp=26.9 C at 403771 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #203 -> temp=29.9 C at 405778 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #204 -> temp=28.2 C at 407786 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #205 -> temp=32.9 C at 409793 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #206 -> temp=27.5 C at 411801 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #207 -> temp=22.6 C at 413809 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #208 -> temp=30.0 C at 415816 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #209 -> temp=20.9 C at 417824 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #210 -> temp=22.3 C at 419831 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #211 -> temp=32.1 C at 421839 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #212 -> temp=24.8 C at 423846 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #213 -> temp=34.9 C at 425854 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #214 -> temp=34.9 C at 427861 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #215 -> temp=20.0 C at 429869 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #216 -> temp=29.2 C at 431876 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #217 -> temp=27.9 C at 433884 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #218 -> temp=21.2 C at 435891 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #219 -> temp=23.2 C at 437899 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #220 -> temp=32.8 C at 439906 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #221 -> temp=32.1 C at 441914 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #222 -> temp=20.9 C at 443921 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #223 -> temp=28.0 C at 445929 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #224 -> temp=25.6 C at 447937 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #225 -> temp=26.6 C at 449944 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #226 -> temp=31.6 C at 451952 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #227 -> temp=26.6 C at 453959 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #228 -> temp=27.1 C at 455967 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #229 -> temp=23.1 C at 457974 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #230 -> temp=25.1 C at 459982 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #231 -> temp=25.5 C at 461990 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #232 -> temp=22.7 C at 463997 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #233 -> temp=26.9 C at 466005 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #234 -> temp=25.5 C at 468012 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #235 -> temp=23.1 C at 470020 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #236 -> temp=24.3 C at 472027 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #237 -> temp=34.8 C at 474035 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #238 -> temp=24.3 C at 476042 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #239 -> temp=25.1 C at 478050 ms, actual period = 2008 ms, error = +8 ms

Sensor cycle #240 -> temp=21.4 C at 480057 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #241 -> temp=27.0 C at 482065 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #242 -> temp=26.9 C at 484072 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #243 -> temp=24.0 C at 486080 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #244 -> temp=27.8 C at 488087 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #245 -> temp=26.3 C at 490095 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #246 -> temp=30.1 C at 492102 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #247 -> temp=23.8 C at 494110 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #248 -> temp=25.2 C at 496118 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #249 -> temp=21.9 C at 498125 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #250 -> temp=32.2 C at 500133 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #251 -> temp=29.4 C at 502140 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #252 -> temp=30.2 C at 504148 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #253 -> temp=33.4 C at 506155 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #254 -> temp=20.4 C at 508163 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #255 -> temp=31.5 C at 510170 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #256 -> temp=31.5 C at 512178 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #257 -> temp=25.7 C at 514185 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #258 -> temp=26.5 C at 516193 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #259 -> temp=34.9 C at 518200 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #260 -> temp=22.9 C at 520208 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #261 -> temp=32.6 C at 522215 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #262 -> temp=26.8 C at 524223 ms, actual period = 2008 ms, error = +8 ms
 Sensor cycle #263 -> temp=32.5 C at 526230 ms, actual period = 2007 ms, error = +7 ms
 Sensor cycle #264 -> temp=27.5 C at 528238 ms, actual period = 2008 ms, error = +8 ms

Appendix C.3:

File: *phase4C_scheduler_interference_log.txt*

This log file has the raw serial output for **Phase 4C**, the scheduler timing experiment under CPU interference. In addition to the periodic sensor cycle entries, the log also shows repeated "Busy thread active" messages from the higher-priority interfering thread. Each sensor cycle line reports the measured execution period and the timing error compared to the intended 2000 ms period. Under interference, the actual sensor-thread period shifted upward to approximately **2379–2381 ms**, with timing error around **+379 ms to +381 ms**. This raw dataset supports the main Phase 4 finding that a higher-priority busy thread imposed a **large but repeatable delay** on the lower-priority periodic sensor task while still preserving low jitter.

Content:

Busy thread active

Sensor cycle #50 -> temp=33.7 C at 117668 ms, actual period = 2379 ms, error = +379 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #51 -> temp=33.8 C at 120047 ms, actual period = 2379 ms, error = +379 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #52 -> temp=33.2 C at 122427 ms, actual period = 2380 ms, error = +380 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #53 -> temp=22.8 C at 124807 ms, actual period = 2380 ms, error = +380 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #54 -> temp=21.3 C at 127186 ms, actual period = 2379 ms, error = +379 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #55 -> temp=33.2 C at 129566 ms, actual period = 2380 ms, error = +380 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #56 -> temp=23.5 C at 131946 ms, actual period = 2380 ms, error = +380 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #57 -> temp=22.6 C at 134325 ms, actual period = 2379 ms, error = +379 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #58 -> temp=21.6 C at 136704 ms, actual period = 2379 ms, error = +379 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #59 -> temp=20.8 C at 139084 ms, actual period = 2380 ms, error = +380 ms

Busy thread active

Busy thread active
Busy thread active
Sensor cycle #60 -> temp=29.5 C at 141464 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #61 -> temp=23.6 C at 143844 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #62 -> temp=22.4 C at 146223 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #63 -> temp=23.8 C at 148603 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #64 -> temp=30.3 C at 150982 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #65 -> temp=25.7 C at 153362 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #66 -> temp=23.7 C at 155743 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #67 -> temp=23.7 C at 158123 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #68 -> temp=29.4 C at 160503 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #69 -> temp=31.3 C at 162882 ms, actual period = 2379 ms, error = +379 ms
Busy thread active

Busy thread active
Busy thread active
Sensor cycle #70 -> temp=26.2 C at 165263 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #71 -> temp=25.6 C at 167643 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #72 -> temp=26.4 C at 170022 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #73 -> temp=31.7 C at 172403 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #74 -> temp=30.2 C at 174783 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #75 -> temp=21.8 C at 177164 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #76 -> temp=24.8 C at 179544 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #77 -> temp=24.1 C at 181924 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #78 -> temp=24.7 C at 184304 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #79 -> temp=20.9 C at 186685 ms, actual period = 2381 ms, error = +381 ms
Busy thread active

Busy thread active
Busy thread active
Sensor cycle #80 -> temp=32.3 C at 189065 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #81 -> temp=33.6 C at 191446 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #82 -> temp=26.9 C at 193827 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #83 -> temp=28.6 C at 196208 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #84 -> temp=30.1 C at 198589 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #85 -> temp=33.8 C at 200969 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #86 -> temp=34.3 C at 203350 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #87 -> temp=26.0 C at 205731 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #88 -> temp=24.0 C at 208112 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #89 -> temp=30.5 C at 210492 ms, actual period = 2380 ms, error = +380 ms
Busy thread active

Busy thread active
Busy thread active
Sensor cycle #90 -> temp=34.5 C at 212872 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #91 -> temp=25.1 C at 215252 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #92 -> temp=23.5 C at 217633 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #93 -> temp=27.0 C at 220014 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #94 -> temp=27.4 C at 222395 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #95 -> temp=21.1 C at 224775 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #96 -> temp=34.1 C at 227155 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #97 -> temp=30.4 C at 229535 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #98 -> temp=30.4 C at 231916 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #99 -> temp=27.8 C at 234296 ms, actual period = 2380 ms, error = +380 ms
Busy thread active

Busy thread active
Busy thread active
Sensor cycle #100 -> temp=25.6 C at 236676 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #101 -> temp=32.2 C at 239057 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #102 -> temp=32.7 C at 241436 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #103 -> temp=24.3 C at 243816 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #104 -> temp=21.1 C at 246196 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #105 -> temp=34.8 C at 248575 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #106 -> temp=21.5 C at 250955 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #107 -> temp=23.5 C at 253336 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #108 -> temp=23.2 C at 255716 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #109 -> temp=31.0 C at 258095 ms, actual period = 2379 ms, error = +379 ms
Busy thread active

Busy thread active
Busy thread active
Sensor cycle #110 -> temp=26.9 C at 260475 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #111 -> temp=26.4 C at 262855 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #112 -> temp=32.0 C at 265234 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #113 -> temp=30.6 C at 267613 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #114 -> temp=32.8 C at 269992 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #115 -> temp=22.0 C at 272372 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #116 -> temp=31.6 C at 274752 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #117 -> temp=23.3 C at 277132 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #118 -> temp=28.9 C at 279512 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #119 -> temp=26.6 C at 281893 ms, actual period = 2381 ms, error = +381 ms
Busy thread active

Busy thread active
Busy thread active
Sensor cycle #120 -> temp=21.1 C at 284272 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #121 -> temp=34.5 C at 286650 ms, actual period = 2378 ms, error = +378 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #122 -> temp=34.4 C at 289030 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #123 -> temp=34.8 C at 291409 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #124 -> temp=34.1 C at 293789 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #125 -> temp=28.5 C at 296168 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #126 -> temp=27.8 C at 298548 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #127 -> temp=23.2 C at 300926 ms, actual period = 2378 ms, error = +378 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #128 -> temp=26.1 C at 303305 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #129 -> temp=22.1 C at 305684 ms, actual period = 2379 ms, error = +379 ms
Busy thread active

Busy thread active
Busy thread active
Sensor cycle #130 -> temp=31.9 C at 308062 ms, actual period = 2378 ms, error = +378 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #131 -> temp=34.9 C at 310441 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #132 -> temp=32.0 C at 312821 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #133 -> temp=32.7 C at 315202 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #134 -> temp=23.6 C at 317582 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #135 -> temp=20.5 C at 319962 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #136 -> temp=26.0 C at 322341 ms, actual period = 2379 ms, error = +379 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #137 -> temp=34.2 C at 324722 ms, actual period = 2381 ms, error = +381 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #138 -> temp=29.1 C at 327102 ms, actual period = 2380 ms, error = +380 ms
Busy thread active
Busy thread active
Busy thread active
Sensor cycle #139 -> temp=26.8 C at 329482 ms, actual period = 2380 ms, error = +380 ms
Busy thread active

Busy thread active

Busy thread active

Sensor cycle #140 -> temp=34.0 C at 331861 ms, actual period = 2379 ms, error = +379 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #141 -> temp=27.9 C at 334240 ms, actual period = 2379 ms, error = +379 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #142 -> temp=31.7 C at 336621 ms, actual period = 2381 ms, error = +381 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #143 -> temp=27.0 C at 339001 ms, actual period = 2380 ms, error = +380 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #144 -> temp=31.3 C at 341381 ms, actual period = 2380 ms, error = +380 ms

Busy thread active

Busy thread active

Busy thread active

Sensor cycle #145 -> temp=27.9 C at 343760 ms, actual period = 2379 ms, error = +379 ms

Busy thread active

Appendix D: Additional Screenshots

Appendix D.1:

File: 5_phase3_goal5_build_success.jpg

This screenshot documents successful compilation and linking of the custom BLE temperature node in the phase3_goal5 application directory. The image shows that the build completed without fatal errors and that the Zephyr toolchain generated the expected firmware output for the nrf52840dk/nrf52840 target. This figure is useful as evidence that the custom application reached a stable **build-success milestone** before flashing and runtime validation.

```

[8/8] Linking C executable zephyr\zephyr.elf
Memory region      Used Size  Region Size  %age Used
FLASH:             185528 B      1 MB      18.06%
RAM:                29428 B      256 KB      11.23%
IDT_LIST:           0 GB        32 KB       0.00%
Generating files from C:\Users\feran\zephyrproject\phase3_goal5\build\zephyr\zephyr.elf for board: nrf52840dk/nrf52840
PS C:\Users\feran\zephyrproject\phase3_goal5> west flash --runner nrfjprog
-- west flash: rebuilding
ninja: no work to do.
-- west flash: using runner nrfjprog
-- runners.nrfjprog: reset after flashing requested
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 736.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 629.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 911.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
Using board 1050230858
-- runners.nrfjprog: Flashing file: C:\Users\feran\zephyrproject\phase3_goal5\build\zephyr\zephyr.hex
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 736.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 629.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 911.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3087.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3112.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3349.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3160.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3176.
[ ##### ] 0.000s | Erase file - Check image [ ##### ] 0.000s | Check image validity - Initialize device info [ ##### ]
[ ##### ] 0.000s | Check image validity - Check region 0 settings [ ##### ] 0.000s | Check image validity - block 1 of 3 [ ##### ]
[ ##### ] 0.004s | Check image validity - block 2 of 3 [ ##### ] 0.033s | Check image validity - Finished [ ##### ]
[ ##### ] 0.000s | Erase file - Erasing [ ##### ] 0.000s | Erasing non-volatile memory - block 1 of 2 [ ##### ]
[ ##### ] 1.431s | Erasing non-volatile memory - block 2 of 2 [ ##### ] 0.000s | Erasing non-volatile memory - Erase successful [ ##### ]
[ ##### ] 3.003s | Erase file - Done erasing [ ##### ] 0.000s | Check image validity - Initialize device info [ ##### ]
[ ##### ] 0.000s | Check image validity - Check region 0 settings [ ##### ] 2.977s | Check image validity - block 1 of 3 [ ##### ]
[ ##### ] 0.782s | Check image validity - block 2 of 3 [ ##### ] 0.031s | Check image validity - Finished [ ##### ]
[ ##### ] 0.000s | Program file - Programming [ ##### ] 0.000s | Programming image - block 1 of 2 [ ##### ]
[ ##### ] 0.336s | Programming image - block 2 of 2 [ ##### ] 0.000s | Programming image - Write successful [ ##### ]
[ ##### ] 0.691s | Program file - Done programming [ ##### ] 0.000s | Programming image - Verify successful [ ##### ]
[ ##### ] 0.000s | Verify file - Check image [ ##### ] 0.000s | Check image validity - Initialize device info [ ##### ]
[ ##### ] 0.000s | Check image validity - Check region 0 settings [ ##### ] 2.581s | Check image validity - block 1 of 3 [ ##### ]
[ ##### ] 0.972s | Check image validity - block 2 of 3 [ ##### ] 0.032s | Check image validity - Finished [ ##### ]
[ ##### ] 0.000s | Verify file - Verifying [ ##### ] 0.000s | Verifying image - block 1 of 2 [ ##### ]
[ ##### ] 0.345s | Verifying image - block 2 of 2 [ ##### ] 0.000s | Verifying image - Verify successful [ ##### ]

```

Appendix D.2:

File: 6_phase3_goal5_flash_success.jpg

This screenshot documents successful flashing of the custom BLE temperature node onto the nRF52840 Development Kit using west flash --runner nrfjprog. Although the terminal output includes repeated JLinkARM.dll reported error -256 messages, the final status line confirms that the board was flashed successfully. This image is important because it captures a practical implementation detail of the project: the presence of non-fatal runner warnings during flashing, followed by confirmed successful programming of the target board.

```

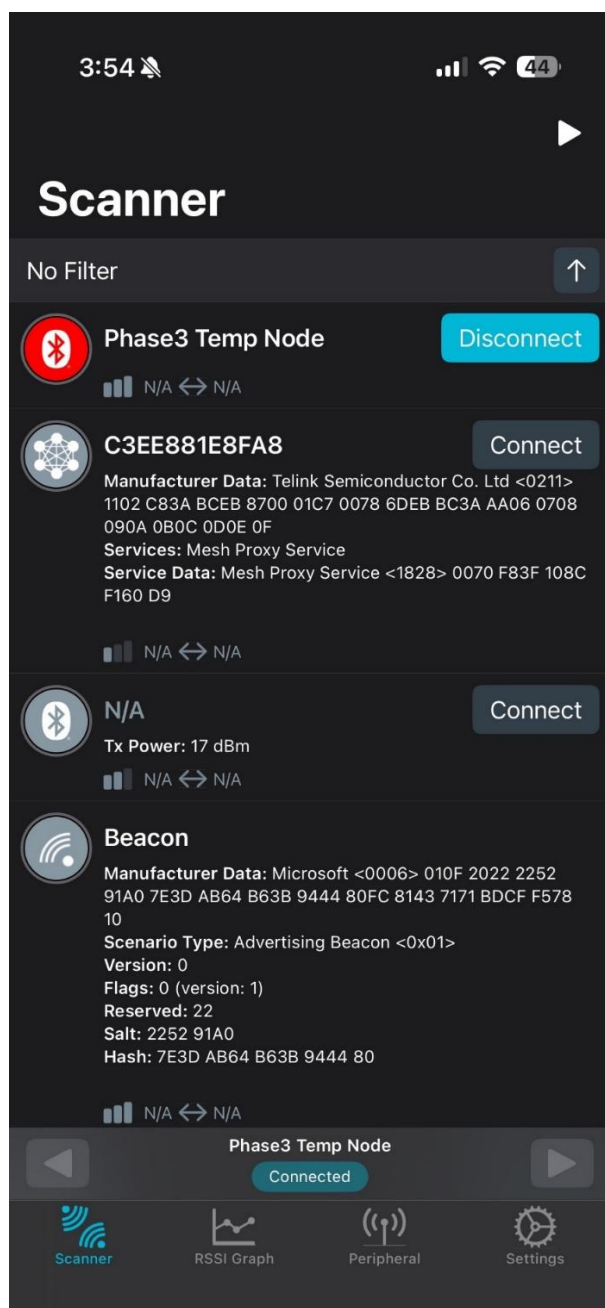
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 911.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
Using board 1050230858
-- runners.nrfjprog: Flashing file: C:\Users\feran\zephyrproject\phase3_goal5\build\zephyr\zephyr.hex
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 736.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 629.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 911.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3087.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3112.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3349.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3160.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3176.
[ ##### ] 0.000s | Erase file - Check image [ ##### ] 0.000s | Check image validity - Initialize device info [ ##### ]
[ ##### ] 0.000s | Check image validity - Check region 0 settings [ ##### ] 0.000s | Check image validity - block 1 of 3 [ ##### ]
[ ##### ] 0.004s | Check image validity - block 2 of 3 [ ##### ] 0.033s | Check image validity - Finished [ ##### ]
[ ##### ] 1.431s | Erasing non-volatile memory - block 2 of 2 [ ##### ] 0.000s | Erasing non-volatile memory - block 1 of 2 [ ##### ]
[ ##### ] 3.003s | Erase file - Done erasing [ ##### ] 0.000s | Erasing non-volatile memory - Erase successful [ ##### ]
[ ##### ] 0.000s | Program file - Checking image [ ##### ] 0.000s | Check image validity - Initialize device info [ ##### ]
[ ##### ] 0.000s | Check image validity - Check region 0 settings [ ##### ] 2.077s | Check image validity - block 1 of 3 [ ##### ]
[ ##### ] 0.782s | Check image validity - block 2 of 3 [ ##### ] 0.031s | Check image validity - Finished [ ##### ]
[ ##### ] 0.000s | Program file - Programming [ ##### ] 0.000s | Programming image - block 1 of 2 [ ##### ]
[ ##### ] 0.336s | Programming image - block 2 of 2 [ ##### ] 0.000s | Programming image - Write successful [ ##### ]
[ ##### ] 0.691s | Program file - Done programming [ ##### ]
[ ##### ] 0.000s | Verify file - Check image [ ##### ] 0.000s | Check image validity - Initialize device info [ ##### ]
[ ##### ] 0.000s | Check image validity - Check region 0 settings [ ##### ] 2.581s | Check image validity - block 1 of 3 [ ##### ]
[ ##### ] 0.972s | Check image validity - block 2 of 3 [ ##### ] 0.032s | Check image validity - Finished [ ##### ]
[ ##### ] 0.000s | Verify file - Verifying [ ##### ] 0.000s | Verifying image - block 1 of 2 [ ##### ]
[ ##### ] 0.345s | Verifying image - block 2 of 2 [ ##### ] 0.000s | Verifying image - Verify successful [ ##### ]
[ ##### ] 0.708s | Verify file - Done verifying [ ##### ]
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 736.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 629.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 911.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3139.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3087.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3112.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3349.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3160.
[error] [SeggerBackend] - JLinkARM.dll reported error -256 at line 3176.
Applying system reset.
Run.
-- runners.nrfjprog: Board(s) with serial number(s) 1050230858 flashed successfully.

```

Appendix D.3:

File: *8_phase3_goal5_scan_result.jpeg*

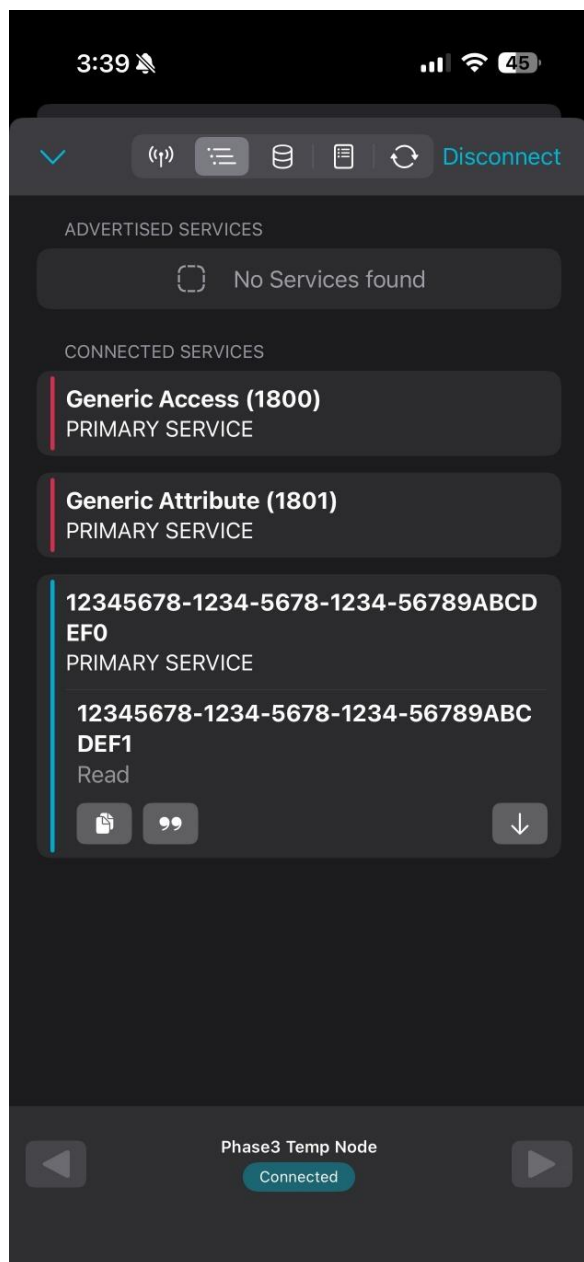
This screenshot from nRF Connect shows the custom BLE device “**Phase3 Temp Node**” appearing in the scanner and connected successfully to the mobile client. The image serves as evidence that the custom Zephyr application was advertising correctly and could be discovered and connected to from the phone. This was one of the key validation results in Phase 3 because it confirmed that the project had progressed from a working embedded application into a working **wireless BLE peripheral**.



Appendix D.4:

File: *10_phase3_goal5_custom_service.jpeg*

This screenshot from nRF Connect displays the custom BLE service and readable characteristics exposed by the Phase3 Temp Node. In addition to the standard Generic Access and Generic Attribute services, the image shows the custom 128-bit primary service UUID and the custom readable characteristic UUID. This figure is important because it provides visual confirmation that the application-level data path was implemented correctly: the board not only advertised and connected but also exposed a **project-specific GATT service and readable characteristic** to the client.

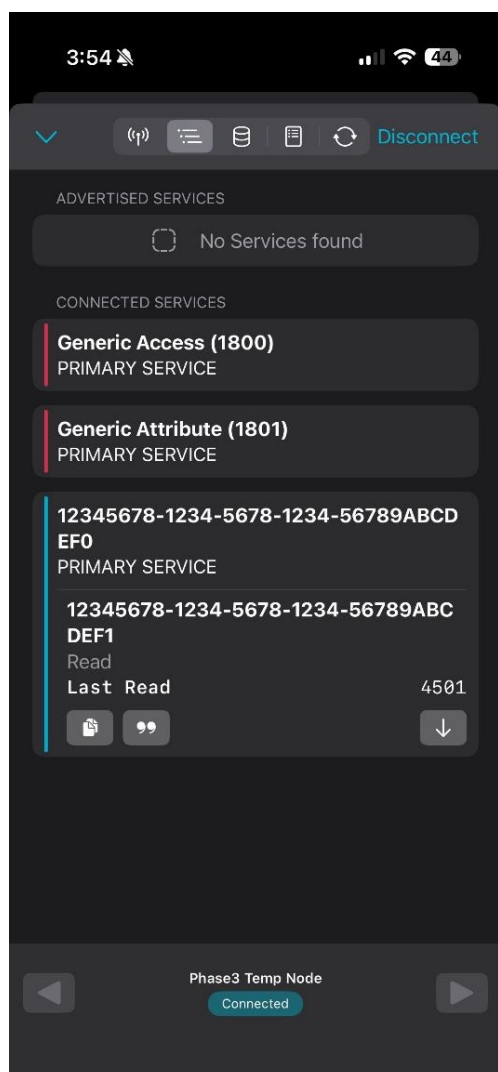


Appendix D.5:

File: *11_phase3_goal5_temp_read1.jpeg*

This screenshot from nRF Connect shows a successful read of the custom BLE characteristic exposed by the Phase3 Temp Node. In addition to the standard Generic Access and Generic Attribute services, the image shows the custom primary service UUID and the readable custom characteristic UUID. The field labeled **“Last Read”** displays the returned value **1301**, providing direct evidence that the mobile client was able to access application-generated data from the board through the custom GATT characteristic.

This figure is important because it documents one of the key functional validation steps in Phase 3: the custom BLE node was not only advertising and connecting successfully but was also serving readable characteristic data to the client application. It therefore supports the report’s claim that BLE data transmission from the Zephyr application to the phone was successfully implemented.

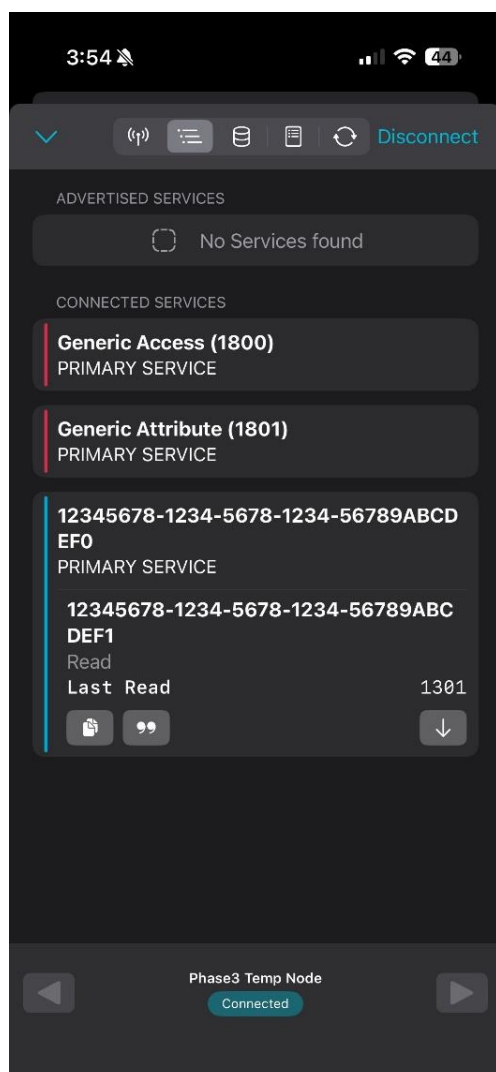


Appendix D.6:

File: *12_phase3_goal5_temp_read2.jpeg*

This screenshot from nRF Connect shows a second successful read of the same custom BLE characteristic exposed by the Phase3 Temp Node. The custom primary service UUID and readable characteristic UUID are again visible, and the “**Last Read**” field now displays the value **4501**. The difference between this value and the earlier read proves that the characteristic was not returning a fixed hardcoded response; instead, it was providing updated application-side data generated on the board.

This figure is especially valuable because it serves as evidence of **changing characteristic values across repeated client reads**. Together with the earlier screenshot, it confirms that the custom BLE node supported live data access rather than only static service visibility. This directly strengthens the report’s discussion of the custom BLE temperature node and the success of Phase 3 system implementation.



Appendix D.7:

File: *Power_Measurement_Setup.jpeg*

This image shows the physical hardware arrangement used during the attempted **Phase 4 power-consumption experiment**. The setup includes the Nordic nRF52840 Development Kit connected with the Nordic Power Profiler Kit II (PPK2). This figure is useful as documentation that power profiling was genuinely tried as part of the project. Even though the resulting current measurements were later judged unreliable and excluded from formal quantitative analysis, this image provides evidence of the instrumentation effort and supports the report's statement that the power experiment was **tried but not finished**.

